

Data Structures & Algorithms + Coding Interview Preparation Masterclass

Master DSA from Scratch | Crack Coding Interviews | Learn Problem Solving | LeetCode | Coding Patterns | Build Strong Logical Thinking

Chapter 1: Introduction to DSA

- **Learning Objective:** Understand what data structures and algorithms (DSA) are and why they are essential in software engineering interviews 【2†L22-L25】. Learn the role of algorithms in problem solving and code efficiency. Gain familiarity with the coding interview process and mindset.
- **Skills Covered:** Problem-solving mindset; basic algorithmic thinking; analysis of algorithm efficiency (Big-O, Big-Theta, Big-Omega) 【18†L52-L60】; interview etiquette basics.
- **Prerequisites:** Basic programming knowledge (variables, loops, functions).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 6
- **Lesson 1.1: What Is DSA?**
 - Difficulty Level: Beginner
 - Learning Outcome: Understand the definition and importance of data structures and algorithms; recognize why DSA is a “staple” of tech interviews 【2†L22-L25】.
 - Topics Covered: Definition of DSA, examples of data structures vs. algorithms, role in efficient programming.
 - Step-by-Step Algorithm Explanation: N/A (conceptual lesson).
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: Diagrams of simple data structures (array, tree) vs. flowchart of algorithm.
 - Practical Exercises: Identify data structures used in everyday applications (e.g., array in photo gallery).
 - Mini Assignment: Write a short paragraph explaining the difference between data structure and algorithm.
 - Common Mistakes: Confusing data structures with algorithms; skipping examples.
 - Interview Questions: “Explain why arrays and linked lists are different,” “What is an algorithm?”

- Quiz Questions: Quick MCQ on DSA definitions.
- **Lesson 1.2: Why DSA?**
 - Difficulty Level: Beginner
 - Learning Outcome: Explain why DSA proficiency yields high ROI for interviews 【2†L22-L25】 ; appreciate how proper data structures impact efficiency.
 - Topics Covered: Benefits of good DSA knowledge, real-world examples (e.g., search/organize large data).
 - Step-by-Step Algorithm Explanation: N/A (discuss examples qualitatively).
 - Time Complexity: Basic idea (reduce from $O(n^2)$ to $O(n \log n)$ through better structures).
 - Space Complexity: N/A (conceptual).
 - Dry Run: Compare solving a problem with vs. without using an efficient DSA.
 - Visual Explanation: Graph showing performance differences of algorithms.
 - Practical Exercises: List three everyday problems that benefit from efficient data structures (e.g., phone contact lookup).
 - Mini Assignment: Research and list why tech companies emphasize DSA in interviews.
 - Common Mistakes: Underestimating the impact of DSA on code performance.
 - Interview Questions: “Why do we learn sorting and searching algorithms in school?”
 - Quiz Questions: True/False on statements about DSA importance.
- **Lesson 1.3: How Coding Interviews Work**
 - Difficulty Level: Beginner
 - Learning Outcome: Learn the structure of technical interviews (phone screen, whiteboard, online coding) and evaluation criteria.
 - Topics Covered: Types of interview rounds, coding question format, problem-solving approach steps.
 - Step-by-Step Algorithm Explanation: N/A (process explanation).
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: Walk through a mock interview question flow (understand, propose, code, test).
 - Visual Explanation: Timeline diagram of interview stages.
 - Practical Exercises: Practice explaining your thought process aloud on a simple problem.
 - Mini Assignment: Document a good problem-solving framework (e.g., clarify requirements, think aloud).

- Common Mistakes: Jumping to code without planning; not asking clarifying questions.
- Interview Questions: “What are the typical rounds in a SDE interview?”
- Quiz Questions: “Which step comes first: writing code or clarifying input constraints?”

- **Lesson 1.4: Problem-Solving Mindset**

- Difficulty Level: Beginner
- Learning Outcome: Cultivate a systematic approach: understand problem, identify data structures, devise algorithms, analyze complexity.
- Topics Covered: Active problem-solving techniques, break-down strategies, pseudocode benefits, pattern recognition.
- Step-by-Step Algorithm Explanation: N/A (meta-level lesson).
- Time Complexity: N/A.
- Space Complexity: N/A.
- Dry Run: Example: solving “find first unique character” with step-by-step reasoning.
- Visual Explanation: Flowchart of problem-solving steps.
- Practical Exercises: Outline solution approach for a given problem in writing before coding.
- Mini Assignment: Take a known problem and write down each step of solution process (e.g., LeetCode “Two Sum”).
- Common Mistakes: Overlooking simpler solutions; not validating edge cases.
- Interview Questions: “What do you do when you’re stuck on a problem?”
- Quiz Questions: Multiple choice on best first step (e.g., “Check examples” vs. “Write code immediately”).

- **Lesson 1.5: Time & Space Complexity**

- Difficulty Level: Beginner
- Learning Outcome: Grasp the concepts of time and space complexity; learn how to compute and compare algorithm efficiency.
- Topics Covered: Concept of input size (n), counting operations; big-O notation basics.
- Step-by-Step Algorithm Explanation: Simple example (loop through array and count operations).
- Time Complexity: $O(n)$ example, explanation of best/worst cases.
- Space Complexity: $O(1)$, $O(n)$ examples (temporary variables vs. arrays).
- Dry Run: Analyze a simple loop (for i in array) and calculate operations.
- Visual Explanation: Graph showing $O(n)$ vs. $O(n^2)$ growth rates.

- Practical Exercises: Determine complexities of given code snippets (one loop, nested loops).
- Mini Assignment: Find complexities of common functions (e.g., sorting the array with built-in sort).
- Common Mistakes: Forgetting constant factors (ignoring $2n$ vs. n), mixing up best/worst case.
- Interview Questions: “What is the time complexity of binary search?”
- Quiz Questions: Provide code and ask to state its big-O time.
- **Lesson 1.6: Big-O, Big-Theta, Big-Omega**
 - Difficulty Level: Beginner
 - Learning Outcome: Differentiate between worst-case (Big-O), best-case (Big-Omega), and tight (Big-Theta) bounds 【18†L52-L60】.
 - Topics Covered: Definitions of O , Ω , Θ ; why each matters; examples.
 - Step-by-Step Algorithm Explanation: Derive notation for simple functions (e.g., $f(n)=3n^2+2n+5$).
 - Time Complexity: Explanation of upper and lower bounds.
 - Space Complexity: Analogy (worst vs. best memory usage).
 - Dry Run: Compare best/worst scenarios (like binary search best-case $O(1)$, worst-case $O(\log n)$).
 - Visual Explanation: Venn diagram of Big-O, Theta, Omega relationships.
 - Practical Exercises: Classify given functions into O , Ω , Θ categories.
 - Mini Assignment: Analyze example algorithms and label their O , Θ , Ω notations.
 - Common Mistakes: Using big-O for average case erroneously; confusion between Θ and O .
 - Interview Questions: “Explain Big-O vs Theta in your own words.”
 - Quiz Questions: Identify if function belongs to $O(n^2)$ or $\Theta(n^2)$ given context.

Chapter 2: Mathematics for DSA

- **Learning Objective:** Build foundational math skills crucial for algorithm design: number theory (primes, GCD, LCM), modular arithmetic, binary math, fast exponentiation. These are building blocks for many DSA problems.
- **Skills Covered:** Modular arithmetic tricks, fast power, binary operations, number theory (prime checking, Euclid’s GCD) 【32†L32-L39】 ; applying math to improve algorithms.
- **Prerequisites:** Arithmetic operations, basic algebra.
- **Estimated Learning Time:** 5–6 hours.
- **Number of Lessons:** 6
- **Lesson 2.1: Prime Numbers**

- Difficulty Level: Beginner
- Learning Outcome: Learn how to check for primes and generate primes efficiently.
- Topics Covered: Definition of prime, trial division, Sieve of Eratosthenes.
- Step-by-Step Algorithm Explanation: Sieve algorithm pseudocode.
- Time Complexity: $O(n \log \log n)$ for Sieve.
- Space Complexity: $O(n)$.
- Dry Run: Mark primes up to a small number using Sieve.
- Visual Explanation: Diagram of crossing off multiples.
- Practical Exercises: List all primes < 30 ; implement simple prime check function.
- Mini Assignment: Code the Sieve and optimize it.
- Common Mistakes: Off-by-one errors (starting from 2); forgetting to mark multiples correctly.
- Interview Questions: “How do you check if a number is prime?”
- Quiz Questions: Fill-in steps of the Sieve algorithm.
- **Lesson 2.2: GCD (Euclid’s Algorithm)**
 - Difficulty Level: Beginner
 - Learning Outcome: Compute the greatest common divisor (GCD) of two numbers efficiently.
 - Topics Covered: Euclid’s algorithm (recursive and iterative).
 - Step-by-Step Algorithm Explanation: Example of Euclid’s GCD process.
 - Time Complexity: $O(\log(\min(a,b)))$.
 - Space Complexity: $O(1)$ iterative, $O(\log n)$ recursion stack.
 - Dry Run: Find $\text{gcd}(48,18)$ step by step.
 - Visual Explanation: Recursion tree or division steps.
 - Practical Exercises: Compute GCDs of sample pairs.
 - Mini Assignment: Extend GCD to compute LCM using formula $\text{LCM}(a,b) = a*b/\text{GCD}(a,b)$.
 - Common Mistakes: Not handling zero correctly.
 - Interview Questions: “Why does Euclid’s algorithm work?”
 - Quiz Questions: Trace Euclid’s algorithm on given inputs.
- **Lesson 2.3: LCM (Least Common Multiple)**
 - Difficulty Level: Beginner
 - Learning Outcome: Use GCD to compute LCM of two numbers.
 - Topics Covered: Definition of LCM, formula via GCD.
 - Step-by-Step Algorithm Explanation: Compute $\text{LCM}(a,b) = a/\text{GCD}(a,b)*b$.
 - Time Complexity: $O(\log n)$.

- Space Complexity: $O(1)$.
- Dry Run: Calculate LCM(4,6) using GCD.
- Visual Explanation: Number line showing multiples.
- Practical Exercises: Find LCM for given pairs using code.
- Mini Assignment: Generalize to multiple numbers (LCM of array).
- Common Mistakes: Overflow when multiplying a and b; not using GCD first.
- Interview Questions: “How is LCM related to GCD?”
- Quiz Questions: Quick calculation of LCM from GCD.

• **Lesson 2.4: Modular Arithmetic**

- Difficulty Level: Beginner
- Learning Outcome: Understand operations under modulo; use mod to handle large numbers.
- Topics Covered: Modulo properties, modular addition/multiplication, avoiding overflow.
- Step-by-Step Algorithm Explanation: Compute $(a+b)\%m$, $(a*b)\%m$ correctly.
- Time Complexity: $O(1)$.
- Space Complexity: $O(1)$.
- Dry Run: Example of modular addition (e.g., $(9+6)\%10$).
- Visual Explanation: Clock arithmetic analogy.
- Practical Exercises: Compute powers mod m; implement modular multiplication.
- Mini Assignment: Solve “power of large exponent mod m” via repeated squaring.
- Common Mistakes: Forgetting to mod after each multiplication.
- Interview Questions: “Why do we use modulo in algorithms?”
- Quiz Questions: Simple mod operation questions.

• **Lesson 2.5: Binary Mathematics (Bitwise Operations)**

- Difficulty Level: Beginner
- Learning Outcome: Apply basic bitwise operations (AND, OR, XOR, NOT, shifts) in problem solving.
- Topics Covered: Bitwise operators, bit masks, shifting, two’s complement.
- Step-by-Step Algorithm Explanation: Example of using XOR to swap values or AND to check even/odd.
- Time Complexity: $O(1)$.
- Space Complexity: $O(1)$.
- Dry Run: Swap two numbers using XOR step by step.
- Visual Explanation: Binary representation of numbers showing bitwise AND, OR.
- Practical Exercises: Determine if a number is a power of two using bits.

- Mini Assignment: Implement function to count set bits in an integer.
- Common Mistakes: Confusing bitwise XOR vs logical XOR; not using unsigned types for shifts.
- Interview Questions: “How to check if the i th bit is set in a number?”
- Quiz Questions: Identify bitwise results (e.g., $5 \& 3$, $5 | 2$).
- **Lesson 2.6: Fast Exponentiation (Binary Exponentiation)**
 - Difficulty Level: Beginner
 - Learning Outcome: Compute large powers efficiently using the divide-and-conquer method.
 - Topics Covered: Exponentiation by squaring, handling powers of integers mod m .
 - Step-by-Step Algorithm Explanation: Example computing 2^{10} by repeatedly squaring.
 - Time Complexity: $O(\log \text{exponent})$.
 - Space Complexity: $O(1)$ iterative or $O(\log n)$ recursion stack.
 - Dry Run: Calculate 3^{13} with binary exponentiation.
 - Visual Explanation: Exponentiation tree showing squaring steps.
 - Practical Exercises: Implement fast power function; compare with naive multiplication.
 - Mini Assignment: Use fast exponentiation to compute large Fibonacci via matrix power.
 - Common Mistakes: Off-by-one in exponent loop; not applying mod at each multiplication.
 - Interview Questions: “How do you compute $a^n \bmod m$ efficiently?”
 - Quiz Questions: Trace steps of binary exponentiation for small inputs.

Chapter 3: Recursion

- **Learning Objective:** Master recursion as a problem-solving tool: write functions that call themselves to break problems into subproblems. Learn how to analyze and debug recursive algorithms.
- **Skills Covered:** Writing recursive solutions; understanding call stack; converting recursion to iteration; identifying base and recursive cases.
- **Prerequisites:** Chapter 1 content, basic math, ability to follow function calls.
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 4
- **Lesson 3.1: Introduction to Recursion**
 - Difficulty Level: Beginner
 - Learning Outcome: Understand the concept of recursion and how recursive calls unfold.

- Topics Covered: Base case vs recursive case, stack frames, simple examples (factorial, Fibonacci).
- Step-by-Step Algorithm Explanation: Factorial(n): if $n==1$ return 1; else return $n*\text{factorial}(n-1)$.
- Time Complexity: $O(n)$ for factorial or linear recursion; Fibonacci naive $O(2^n)$.
- Space Complexity: $O(n)$ (due to recursion stack).
- Dry Run: Trace factorial(4) through calls.
- Visual Explanation: Call stack diagram for factorial(4).
- Practical Exercises: Write a recursive function for factorial and compute values.
- Mini Assignment: Implement a recursive function to find the nth Fibonacci number (inefficient version).
- Common Mistakes: Missing base case; infinite recursion.
- Interview Questions: “What is recursion and when would you use it?”
- Quiz Questions: Identify base cases in given recursive functions.
- **Lesson 3.2: Divide and Conquer Recursion**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use recursion in divide-and-conquer algorithms (e.g., binary search, merge sort).
 - Topics Covered: Splitting problem into halves, example of binary search and merge sort recursion.
 - Step-by-Step Algorithm Explanation: Show binary search pseudocode.
 - Time Complexity: $O(\log n)$ for binary search, $O(n \log n)$ for merge sort.
 - Space Complexity: $O(\log n)$ recursion stack (binary search), $O(n)$ additional for merge sort.
 - Dry Run: Binary search on sorted list of size 8.
 - Visual Explanation: Tree diagram of divide step (array split).
 - Practical Exercises: Implement binary search recursively; apply merge sort to a small array.
 - Mini Assignment: Write recursive merge function for two sorted halves.
 - Common Mistakes: Off-by-one mid index, not reducing search space.
 - Interview Questions: “Why is merge sort $O(n \log n)$?”
 - Quiz Questions: Determine number of recursion levels for merge sort on 16 elements.
- **Lesson 3.3: Recursion with Backtracking**
 - Difficulty Level: Intermediate
 - Learning Outcome: See how recursion can explore multiple branches (backtracking) with examples.

- Topics Covered: Backtracking concept (see next chapter in depth); simple example of generating subsets.
- Step-by-Step Algorithm Explanation: Recursively building subsets of a set.
- Time Complexity: $O(2^n)$ for generating all subsets.
- Space Complexity: $O(n)$ stack plus output size.
- Dry Run: Generate subsets of $[a, b]$ step by step.
- Visual Explanation: Tree of recursive calls (branching into include/exclude).
- Practical Exercises: Write recursive code to generate all binary strings of length n .
- Mini Assignment: Explore a small backtracking problem (like simple maze pathfinding).
- Common Mistakes: Not returning out of recursive paths properly, forgetting to undo choices.
- Interview Questions: “Explain how recursion is used in backtracking.”
- Quiz Questions: Give number of subsets of a 4-element set.
- **Lesson 3.4: Advanced Recursion Examples**
 - Difficulty Level: Advanced
 - Learning Outcome: Tackle complex recursive problems (e.g., Tower of Hanoi, recursive tree traversals).
 - Topics Covered: Tower of Hanoi, recursion on trees (preorder, inorder examples).
 - Step-by-Step Algorithm Explanation: Solve Tower of Hanoi with n disks.
 - Time Complexity: $O(2^n)$ for Tower of Hanoi, $O(n)$ for tree traversals.
 - Space Complexity: $O(n)$ (stack depth).
 - Dry Run: Move 3-disk Tower of Hanoi.
 - Visual Explanation: Recursive call tree for tree traversal.
 - Practical Exercises: Implement inorder traversal of a binary tree recursively.
 - Mini Assignment: Write recursive solution for Lucas numbers or similar.
 - Common Mistakes: Stack overflow (deep recursion); missing base case in complex recursion.
 - Interview Questions: “How many moves does Tower of Hanoi take for n disks?”
 - Quiz Questions: Sketch the call graph for a simple recursive algorithm.

Chapter 4: Backtracking

- **Learning Objective:** Master the backtracking paradigm: generate permutations/combinations and solve constraint problems (e.g., N-Queens, Sudoku) by exploring solution space.
- **Skills Covered:** Recursive backtracking, pruning search space, solving combinatorial problems, recursion with multiple recursive calls.

- **Prerequisites:** Recursion (Chapter 3), understanding of arrays or matrices.
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 3
- **Lesson 4.1: Introduction to Backtracking**
 - Difficulty Level: Intermediate
 - Learning Outcome: Understand backtracking as recursive search with undoing (“backtracking”) choices.
 - Topics Covered: Backtracking vs pure recursion, recursion trees, simple example (generate combinations).
 - Step-by-Step Algorithm Explanation: Backtracking outline (choose, recurse, un-choose).
 - Time Complexity: Exponential in general (branching factor^{depth}).
 - Space Complexity: $O(n)$ stack plus solution space.
 - Dry Run: Generate all subsets of [1,2,3] using backtracking approach.
 - Visual Explanation: Recursion tree with pruning indicated.
 - Practical Exercises: Write code for generating all subsets or combinations of given length.
 - Mini Assignment: Implement combination sum (backtracking subset sum).
 - Common Mistakes: Not reverting changes on return (not “backtracking”).
 - Interview Questions: “What is backtracking? Give an example.”
 - Quiz Questions: Describe the difference between recursion and backtracking.
- **Lesson 4.2: Permutations and Combinations**
 - Difficulty Level: Intermediate
 - Learning Outcome: Generate all permutations and combinations of a set recursively.
 - Topics Covered: Heap’s algorithm for permutations, recursive combination generation.
 - Step-by-Step Algorithm Explanation: Example of generating permutations of [1,2,3].
 - Time Complexity: $O(n!)$ for permutations, $O(2^n)$ for subsets.
 - Space Complexity: $O(n)$ stack plus output.
 - Dry Run: Permutations of [a,b,c] step by step.
 - Visual Explanation: Tree of recursion swapping elements.
 - Practical Exercises: Code to print all permutations of a string.
 - Mini Assignment: Write a function to generate k-combinations of n.
 - Common Mistakes: Treating array indices incorrectly, duplicate results without handling.

- Interview Questions: “How many permutations are there of n elements?”
- Quiz Questions: Trace the first two recursive calls of permutation generation.
- **Lesson 4.3: Constraint Problems (N-Queens, Sudoku)**
 - Difficulty Level: Advanced
 - Learning Outcome: Apply backtracking to solve constraint satisfaction problems.
 - Topics Covered: N-Queens (place queens row by row), Sudoku solver outline.
 - Step-by-Step Algorithm Explanation: Solve 4-Queens board, showing row-by-row placement.
 - Time Complexity: Approx $O(n!)$, significantly reduced by pruning.
 - Space Complexity: $O(n^2)$ for board, plus recursion stack.
 - Dry Run: Place queens on 4x4 board with no conflicts.
 - Visual Explanation: Chessboard diagrams at each step.
 - Practical Exercises: Code a backtracking solver for Sudoku or simple N-Queens.
 - Mini Assignment: Optimize N-Queens by using columns/diagonals arrays for fast conflict check.
 - Common Mistakes: Not checking all constraints, forgetting to remove queen (backtrack).
 - Interview Questions: “Why is N-Queens considered backtracking?”
 - Quiz Questions: How many solutions does 8-Queens have? (Trick question: 92).

Chapter 5: Arrays

- **Learning Objective:** Gain deep proficiency with one of the most fundamental data structures: arrays. Cover array manipulation, problem solving, and common array-based interview questions 【8†L118-L124】.
- **Skills Covered:** Array operations (search, insert, delete, sort); sliding window techniques; two-pointer strategies; implementing array algorithms.
- **Prerequisites:** Basic coding, Chapter 1 concepts.
- **Estimated Learning Time:** 6–7 hours.
- **Number of Lessons:** 7
- **Lesson 5.1: Array Basics and Operations**
 - Difficulty Level: Beginner
 - Learning Outcome: Understand array structure in memory, indexing, and basic operations.
 - Topics Covered: Declaration, access, updating elements, 1D vs 2D arrays.
 - Step-by-Step Algorithm Explanation: N/A (conceptual overview).

- Time Complexity: Access $O(1)$, search $O(n)$.
- Space Complexity: $O(n)$ storage.
- Dry Run: Access and modify elements in sample array.
- Visual Explanation: Memory layout of an array.
- Practical Exercises: Implement functions to find max/min in array.
- Mini Assignment: Write code to rotate an array by k positions.
- Common Mistakes: Index out of bounds; forgetting to handle empty array.
- Interview Questions: “How do you find an element in an array?”
- Quiz Questions: State the time complexity of inserting into middle of an array.
- **Lesson 5.2: Finding Pairs (Two Sum, Two-Pointer)**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve pair-sum problems using two-pointer or hash techniques.
 - Topics Covered: Two-pointer on sorted arrays, hash set for unsorted.
 - Step-by-Step Algorithm Explanation: Solve “Two Sum” by sorting + two pointers or using a map.
 - Time Complexity: $O(n \log n)$ (sort + two pointers) or $O(n)$ with hash.
 - Space Complexity: $O(1)$ (two pointers) or $O(n)$ (hash map).
 - Dry Run: Example: find pair summing to 10 in $[1, 3, 4, 7, 10]$ with two pointers.
 - Visual Explanation: Pointers moving inward from array ends.
 - Practical Exercises: Implement Two Sum with both approaches.
 - Mini Assignment: Given sorted array, count all pairs with $\text{sum} = K$.
 - Common Mistakes: Not sorting before two-pointer; forgetting to check both pointer moves.
 - Interview Questions: “How to find a target sum in array efficiently?”
 - Quiz Questions: Describe steps of two-pointer algorithm for pair sum.
- **Lesson 5.3: Subarray Problems and Kadane’s Algorithm**
 - Difficulty Level: Intermediate
 - Learning Outcome: Find subarrays with maximum sum or other properties using prefix/suffix or Kadane’s algorithm.
 - Topics Covered: Maximum subarray sum (Kadane’s), subarray sum equals k (hash), sliding window.
 - Step-by-Step Algorithm Explanation: Derive Kadane’s DP solution.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: Kadane’s on $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$.
 - Visual Explanation: Graph showing current vs max sum at each index.
 - Practical Exercises: Implement Kadane’s; count subarrays with $\text{sum} = 0$.

- Mini Assignment: Modify Kadane's to return subarray indices.
- Common Mistakes: Reset logic error, negative number handling.
- Interview Questions: "Explain Kadane's algorithm."
- Quiz Questions: Compute max subarray of a small array by hand.
- **Lesson 5.4: Array Partitioning and Rearrangement**
 - Difficulty Level: Intermediate
 - Learning Outcome: Rearrange arrays using partitioning schemes (Dutch National Flag, partition for quicksort).
 - Topics Covered: Partition algorithms, in-place rearrangement.
 - Step-by-Step Algorithm Explanation: Dutch National Flag (three-way partition) for values $<$, $=$, $>$ pivot.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: Partition $[2, 0, 2, 1, 1, 0]$ around 1.
 - Visual Explanation: Coloring diagram moving pivot groups.
 - Practical Exercises: Implement partition function; separate evens/odds in array.
 - Mini Assignment: Use partition to solve "sort colors" problem.
 - Common Mistakes: Wrong swap logic; infinite loops.
 - Interview Questions: "What is Dutch National Flag problem?"
 - Quiz Questions: Identify result of a single partition step on example.
- **Lesson 5.5: Multidimensional Arrays and Matrices**
 - Difficulty Level: Intermediate
 - Learning Outcome: Work with 2D arrays/matrices; solve basic matrix problems (transpose, rotate).
 - Topics Covered: Row-major representation, matrix transposition, matrix rotation by 90° .
 - Step-by-Step Algorithm Explanation: Rotate matrix in layers.
 - Time Complexity: $O(n^2)$ for $N \times N$ matrix.
 - Space Complexity: $O(1)$ in-place (plus $O(n^2)$ if copying).
 - Dry Run: Transpose and rotate a 3×3 matrix.
 - Visual Explanation: Matrix before/after transformations.
 - Practical Exercises: Implement matrix transpose and rotate functions.
 - Mini Assignment: Given an image matrix, rotate it by $k \times 90^\circ$.
 - Common Mistakes: Forgetting to swap all four cells in rotation; out-of-bounds.
 - Interview Questions: "How to rotate an $N \times N$ matrix in place?"
 - Quiz Questions: Trace index mapping during one layer of rotation.
- **Lesson 5.6: Searching and Sorting within Arrays**

- Difficulty Level: Intermediate
- Learning Outcome: Apply built-in and custom search/sort algorithms to solve problems faster.
- Topics Covered: Linear vs binary search in array context; usage of sorting before other operations.
- Step-by-Step Algorithm Explanation: Example binary search in sorted array.
- Time Complexity: $O(\log n)$ search, $O(n \log n)$ sort.
- Space Complexity: $O(1)$ for in-place sort, $O(n)$ for some sorting methods.
- Dry Run: Binary search for 7 in $[1, 3, 5, 7, 9]$.
- Visual Explanation: Halving array segments during binary search.
- Practical Exercises: Use binary search in a sorted array to find leftmost/rightmost index of a value.
- Mini Assignment: Implement selection sort and compare performance to built-in sort.
- Common Mistakes: Not handling even-sized arrays mid, ignoring boundaries in binary search.
- Interview Questions: “Why must an array be sorted for binary search?”
- Quiz Questions: Give mid index for array of size 5.

• Lesson 5.7: Array Practice and Patterns

- Difficulty Level: Intermediate
- Learning Outcome: Consolidate array skills by solving pattern-based problems (sliding window, prefix sum on arrays).
- Topics Covered: Recognize array problem patterns (subarray sums, in-place updates).
- Step-by-Step Algorithm Explanation: Illustration of prefix-sum to answer range queries in array.
- Time Complexity: $O(n)$ preprocessing, $O(1)$ per query **【32†L32-L39】** .
- Space Complexity: $O(n)$ additional for prefix array.
- Dry Run: Build prefix sum of $[2, 4, 6, 8]$ and answer sum query $[1, 3]$.
- Visual Explanation: Array with overlay of prefix sums.
- Practical Exercises: Given array and multiple queries $[L, R]$, compute sums with prefix sums.
- Mini Assignment: Solve “maximum subarray after one deletion” pattern.
- Common Mistakes: Off-by-one indexing in prefix sum array.
- Interview Questions: “How to compute range sum quickly?”
- Quiz Questions: Calculate prefix sum outputs for example array.

Chapter 6: Strings

- **Learning Objective:** Master string operations and problems. Understand that strings are character arrays **【37†L242-L249】**, enabling use of array techniques. Cover string searching, manipulation, and pattern matching.
- **Skills Covered:** String traversal and manipulation; common string problems (palindromes, anagrams); pattern matching fundamentals.
- **Prerequisites:** Arrays (Chapter 5), basic recursion (Chapter 3).
- **Estimated Learning Time:** 5–6 hours.
- **Number of Lessons:** 5
- **Lesson 6.1: String Basics and Operations**
 - Difficulty Level: Beginner
 - Learning Outcome: Learn string representation in memory, basic operations (concatenation, substring, iteration).
 - Topics Covered: Immutable vs mutable strings (language-specific), string indexing and length.
 - Step-by-Step Algorithm Explanation: N/A (conceptual).
 - Time Complexity: Access $O(1)$, concatenation usually $O(n)$.
 - Space Complexity: $O(n)$ for new strings.
 - Dry Run: Extract substring “ell” from “Hello”.
 - Visual Explanation: Memory of char array vs string object.
 - Practical Exercises: Reverse a string, check if palindrome.
 - Mini Assignment: Remove duplicates in a string preserving order.
 - Common Mistakes: Off-by-one in substring, forgetting immutability cost.
 - Interview Questions: “What is the time complexity of string concatenation?”
 - Quiz Questions: Output of given simple string manipulation code.
- **Lesson 6.2: Common String Problems (Anagram, Palindrome)**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve frequently asked problems: check for anagrams, find longest palindrome, etc.
 - Topics Covered: Character counting with hash or arrays, two-pointer for palindrome.
 - Step-by-Step Algorithm Explanation: Check palindrome by two-pointer comparison.
 - Time Complexity: $O(n)$ for both anagram check (with fixed alphabet) and palindrome check.
 - Space Complexity: $O(1)$ or $O(n)$ for counting.
 - Dry Run: Check if “racecar” is a palindrome using left/right pointers.
 - Visual Explanation: Character frequency table example.

- Practical Exercises: Write code to determine if two strings are anagrams.
- Mini Assignment: Given a string, find longest palindromic substring (basic expand-around-center).
- Common Mistakes: Case-sensitivity issues, ignoring non-letter characters.
- Interview Questions: “How to check if two strings are anagrams?”
- Quiz Questions: Identify palindrome or not for sample inputs.
- **Lesson 6.3: String Pattern Matching (Naive, KMP)**
 - Difficulty Level: Advanced
 - Learning Outcome: Learn how to search for a pattern in a text efficiently.
 - Topics Covered: Brute-force string search, introduction to KMP (prefix function).
 - Step-by-Step Algorithm Explanation: Outline KMP prefix table computation.
 - Time Complexity: $O(n)$ for KMP search.
 - Space Complexity: $O(m)$ for prefix array.
 - Dry Run: Example pattern “ABA” in text “ABABABA” with KMP table.
 - Visual Explanation: Prefix (lps) array construction for “ABCAB”.
 - Practical Exercises: Implement KMP; compare with naive search on long strings.
 - Mini Assignment: Solve “find substring indices” problem.
 - Common Mistakes: Computing LPS table incorrectly; infinite loops on mismatch.
 - Interview Questions: “Explain KMP algorithm in brief.”
 - Quiz Questions: Construct prefix table for a sample pattern.
- **Lesson 6.4: Advanced String Manipulations**
 - Difficulty Level: Advanced
 - Learning Outcome: Tackle complex string challenges (string decoding, URLify, regex basics).
 - Topics Covered: In-place string algorithms, wildcard matching (overview).
 - Step-by-Step Algorithm Explanation: E.g., decode “3[a]2[bc]” style patterns.
 - Time Complexity: Depends (often $O(n*m)$ or more for wildcard backtracking).
 - Space Complexity: Auxiliary data structures (stack or recursion).
 - Dry Run: Decode example string with nested patterns.
 - Visual Explanation: Stack-based simulation of decoding.
 - Practical Exercises: Implement URLify (replace spaces), Validate parentheses string.
 - Mini Assignment: Write a function to multiply two large integers given as strings.
 - Common Mistakes: Ignoring edge cases of regex matches; stack underflow.
 - Interview Questions: “How to replace spaces in a string in-place?”

- Quiz Questions: Evaluate given decoded string output.
- **Lesson 6.5: String Practice and Patterns**
 - Difficulty Level: Intermediate
 - Learning Outcome: Reinforce string handling by solving pattern-based string problems (sliding window on strings, prefix/suffix).
 - Topics Covered: Longest substring without repeating characters (sliding window), string hashing idea.
 - Step-by-Step Algorithm Explanation: Sliding window to find longest unique-character substring.
 - Time Complexity: $O(n)$ using sliding window.
 - Space Complexity: $O(\min(n, \text{alphabet}))$.
 - Dry Run: Slide window over "abcabcbb".
 - Visual Explanation: Window boundaries moving over string.
 - Practical Exercises: Implement longest unique substring; rolling hash for fixed-length substring matching.
 - Mini Assignment: Find smallest window in string containing all characters of another string.
 - Common Mistakes: Not moving left pointer correctly; forgetting to shrink window.
 - Interview Questions: "Describe sliding window technique on strings."
 - Quiz Questions: Determine longest non-repeating substring length for a short string.

Chapter 7: Searching

- **Learning Objective:** Cover fundamental search techniques: linear and binary search, including "binary search on answer" technique for optimization problems.
- **Skills Covered:** Linear search (unsorted), binary search (sorted), binary search on function monotonicity, search variations.
- **Prerequisites:** Arrays (Chapter 5), basic loops.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 7.1: Linear Search**
 - Difficulty Level: Beginner
 - Learning Outcome: Implement basic linear search through arrays/lists.
 - Topics Covered: Sequential scanning, finding first/last occurrence, sentinel technique.
 - Step-by-Step Algorithm Explanation: Loop through elements until target is found.
 - Time Complexity: $O(n)$.

- Space Complexity: $O(1)$.
- Dry Run: Search 7 in [2,5,7,1,7].
- Visual Explanation: Linear scan with marker.
- Practical Exercises: Write linear search to find all indices of a key.
- Mini Assignment: Optimize linear search when multiple queries by building a hash map (hybrid technique).
- Common Mistakes: Forgetting to check all elements; off-by-one.
- Interview Questions: “When is linear search preferred over binary search?”
- Quiz Questions: Predict output index of linear search on example array.

• Lesson 7.2: Binary Search

- Difficulty Level: Intermediate
- Learning Outcome: Perform binary search on sorted arrays and understand its invariants.
- Topics Covered: Iterative/recursive binary search, search boundaries, variants (first $\geq$ target, etc).
- Step-by-Step Algorithm Explanation: Example iterative binary search for target in sorted array.
- Time Complexity: $O(\log n)$.
- Space Complexity: $O(1)$.
- Dry Run: Search 8 in [1,3,5,7,9] with low/high pointers.
- Visual Explanation: Halving interval graphic.
- Practical Exercises: Implement binary search to find insert position for target.
- Mini Assignment: Solve “find first bad version” using binary search.
- Common Mistakes: Infinite loop by wrong mid adjustment; not terminating on low>high.
- Interview Questions: “How many comparisons does binary search take on average?”
- Quiz Questions: Steps of binary search for target=6 in [2,4,6,8,10].

• Lesson 7.3: Binary Search on Answer

- Difficulty Level: Advanced
- Learning Outcome: Use binary search to find a value by searching over possible answers (monotonic predicate).
- Topics Covered: Parametric search, typical problems (finding smallest feasible, splitting arrays).
- Step-by-Step Algorithm Explanation: Example: find minimum maximum subarray sum given splits allowed.
- Time Complexity: $O(\log M * f(n))$ where M is search range, $f(n)$ is validation cost.

- Space Complexity: $O(1)$ or as needed by check function.
- Dry Run: Binary search over range [min element, sum of elements] for target largest sum.
- Visual Explanation: Predicate function curve (false→true).
- Practical Exercises: Given array and k, find minimum largest sum when splitting into k subarrays.
- Mini Assignment: Search on answer for “kth smallest element in multiplication table”.
- Common Mistakes: Off-by-one in search bounds; invalid monotonic check.
- Interview Questions: “What is binary search on answer?”
- Quiz Questions: Outline steps to find the k-th root using binary search.

Chapter 8: Sorting

- **Learning Objective:** Learn fundamental sorting algorithms (bubble, insertion, selection) and advanced sorts (merge, quick, heap) 【8†L89-L94】. Understand their trade-offs and when to use each.
- **Skills Covered:** Implement and analyze common sorts; recognize stability and in-place behavior; apply sorting in problem solving (sort+two-pointer, etc).
- **Prerequisites:** Arrays (Chapter 5).
- **Estimated Learning Time:** 6–7 hours.
- **Number of Lessons:** 6
- **Lesson 8.1: Bubble, Selection, Insertion Sort**
 - Difficulty Level: Beginner
 - Learning Outcome: Implement basic $O(n^2)$ sorts; understand simple comparisons/swaps.
 - Topics Covered: Algorithms of bubble, selection, insertion; stability, best vs worst cases.
 - Step-by-Step Algorithm Explanation: Bubble sort passes example on small array.
 - Time Complexity: $O(n^2)$ average and worst.
 - Space Complexity: $O(1)$ (in-place).
 - Dry Run: Perform one pass of insertion sort on [3,1,4,2].
 - Visual Explanation: Animated swaps of bubble sort.
 - Practical Exercises: Code each simple sort; compare steps count.
 - Mini Assignment: Optimize bubble sort by stopping early if sorted.
 - Common Mistakes: Off-by-one in loops, skipping last element.
 - Interview Questions: “When is insertion sort useful?”
 - Quiz Questions: Number of swaps in selection sort of [4,3,2,1].
- **Lesson 8.2: Merge Sort**

- Difficulty Level: Intermediate
- Learning Outcome: Implement merge sort; understand divide-and-conquer for sorting.
- Topics Covered: Recursive splitting, merging sorted subarrays.
- Step-by-Step Algorithm Explanation: Merge sort on [5,2,4,7,1,3].
- Time Complexity: $O(n \log n)$.
- Space Complexity: $O(n)$ additional array for merging.
- Dry Run: Merge [2,5,7] and [1,3,4] into [1,2,3,4,5,7].
- Visual Explanation: Tree diagram splitting array.
- Practical Exercises: Implement merge sort and count inversions.
- Mini Assignment: Given two sorted lists, merge them efficiently.
- Common Mistakes: Indexing errors in merge; missing element at end of merge.
- Interview Questions: “Why is merge sort stable?”
- Quiz Questions: Number of levels in recursion tree for merge sort of 8 elements.
- **Lesson 8.3: Quick Sort**
 - Difficulty Level: Intermediate
 - Learning Outcome: Implement quicksort; grasp partition logic and randomization for performance.
 - Topics Covered: Partition schemes (Lomuto/Hoare), choosing pivots, tail recursion optimization.
 - Step-by-Step Algorithm Explanation: Quick sort [3,6,8,10,1,2,1] with pivot=last element.
 - Time Complexity: Average $O(n \log n)$, worst $O(n^2)$ (if bad pivot) 【18†L112-L120】.
 - Space Complexity: $O(\log n)$ expected recursion stack ($O(n)$ worst).
 - Dry Run: Partition step on [3,6,8,10,1,2,1] pivot=1.
 - Visual Explanation: Partitioning illustration (elements < pivot to left).
 - Practical Exercises: Implement quicksort with random pivot; track comparisons.
 - Mini Assignment: Analyze quicksort on sorted input (worst case).
 - Common Mistakes: Not randomizing pivot; incorrect pointer movement.
 - Interview Questions: “When does quicksort degrade to $O(n^2)$?”
 - Quiz Questions: Final position of pivot after one partition.
- **Lesson 8.4: Heap Sort**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use a heap to sort: build a heap and repeatedly extract max/min.

- Topics Covered: Max-heap structure, heapify operation, in-place heap sort algorithm.
- Step-by-Step Algorithm Explanation: Build max-heap of [3,1,4,2], then sort.
- Time Complexity: $O(n \log n)$.
- Space Complexity: $O(1)$ (in-place).
- Dry Run: Heapify array [3,1,4,2]; then remove max 4.
- Visual Explanation: Binary heap tree before/after heapify.
- Practical Exercises: Implement heap sort using array, and use priority queue as alternative.
- Mini Assignment: Heapify a given list and show intermediate steps.
- Common Mistakes: 1-index vs 0-index confusion in array heap.
- Interview Questions: “Is heap sort stable?” (No).
- Quiz Questions: Parent/child indices for an element at index i .
- **Lesson 8.5: Counting Sort**
 - Difficulty Level: Beginner
 - Learning Outcome: Sort non-negative integers in linear time using counting.
 - Topics Covered: Counting frequency array, prefix sum for positions, stable output generation.
 - Step-by-Step Algorithm Explanation: Counting sort on [4,2,2,8,3,3,1].
 - Time Complexity: $O(n + k)$ where k = range of input values.
 - Space Complexity: $O(n + k)$.
 - Dry Run: Build count array for values 1..8, then cumulative counts.
 - Visual Explanation: Graphical count of occurrences, output filling.
 - Practical Exercises: Implement counting sort for small range data.
 - Mini Assignment: Use counting sort to solve “sort colours” (0,1,2).
 - Common Mistakes: Large k makes it impractical; forgetting stable fill from right.
 - Interview Questions: “When is counting sort useful?”
 - Quiz Questions: Calculate count array for a simple list.
- **Lesson 8.6: Radix Sort**
 - Difficulty Level: Advanced
 - Learning Outcome: Sort integers (or strings) by processing digits or characters (non-comparative sort).
 - Topics Covered: LSD vs MSD radix sort, using stable sort (counting) on digits.
 - Step-by-Step Algorithm Explanation: Radix sort on [170,45,75,90,802,24,2,66] by digit.
 - Time Complexity: $O(d*(n+k))$ where d = number of digits.
 - Space Complexity: $O(n + k)$.
 - Dry Run: Sort example numbers by units, then tens, etc.

- Visual Explanation: Buckets for each digit pass.
- Practical Exercises: Implement radix sort for fixed-digit numbers.
- Mini Assignment: Generalize to sort strings by character.
- Common Mistakes: Not using stable intermediate sort; variable-length digits.
- Interview Questions: “Explain LSD radix sort.”
- Quiz Questions: Order of array after first pass of radix sort.

Chapter 9: Hashing

- **Learning Objective:** Learn hash-based data structures for fast lookup: hash tables, hash sets/maps. Cover collision handling and practical uses (caching, counting).
- **Skills Covered:** Designing hash functions, using hash maps/sets for problems (e.g., Two Sum, frequency count), understanding collision resolution (chaining, open addressing).
- **Prerequisites:** Arrays, basic math (modulo).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 4
- **Lesson 9.1: Hash Tables and Maps**
 - Difficulty Level: Beginner
 - Learning Outcome: Use hash tables (dictionaries/maps) for key-value storage; achieve average $O(1)$ access 【12†L117-L124】.
 - Topics Covered: Hash function concept, buckets, operations: insert/find/delete.
 - Step-by-Step Algorithm Explanation: Not applicable (library usage).
 - Time Complexity: Average $O(1)$ for insert/search, worst $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Insert keys “apple, banana, cherry” into a map.
 - Visual Explanation: Diagram of hash table buckets with chaining.
 - Practical Exercises: Use a map to count frequencies in an array.
 - Mini Assignment: Design a simple hash table using array of vectors (chaining).
 - Common Mistakes: Choosing poor hash (causes clustering); forgetting to handle key collisions.
 - Interview Questions: “How do hash tables handle collisions?”
 - Quiz Questions: State average time for hashmap operations.
- **Lesson 9.2: Hash Sets**
 - Difficulty Level: Beginner

- Learning Outcome: Utilize hash set to check membership in $O(1)$ average time.
- Topics Covered: Set vs. map, typical operations: add, contains, remove.
- Step-by-Step Algorithm Explanation: N/A (conceptual).
- Time Complexity: $O(1)$ average for add/check, worst $O(n)$.
- Space Complexity: $O(n)$.
- Dry Run: Insert numbers into set and test membership.
- Visual Explanation: Hash set with one bucket per value.
- Practical Exercises: Solve “unique elements” problems using sets.
- Mini Assignment: Given an array, remove duplicates using set.
- Common Mistakes: Not handling null keys (language specific).
- Interview Questions: “Why use a set instead of list to check duplicates?”
- Quiz Questions: Outcome of adding duplicate to a set.
- **Lesson 9.3: Collision Resolution Techniques**
 - Difficulty Level: Intermediate
 - Learning Outcome: Understand open addressing (linear probing, double hashing) vs chaining.
 - Topics Covered: Pros/cons of methods, example of rehashing when load factor high.
 - Step-by-Step Algorithm Explanation: Insert in hash table with linear probing.
 - Time Complexity: $O(1)$ average, degrade with clustering.
 - Space Complexity: $O(n)$ table size.
 - Dry Run: Insert elements causing collision and show probes.
 - Visual Explanation: Array slots with probe sequence arrows.
 - Practical Exercises: Implement a simple open-address hash table.
 - Mini Assignment: Compare number of probes vs chaining under same load.
 - Common Mistakes: Infinite loop if table full; forgetting wrap-around.
 - Interview Questions: “What is rehashing in a hash table?”
 - Quiz Questions: Identify next index in linear probing.
- **Lesson 9.4: Hashing in Problem Solving**
 - Difficulty Level: Intermediate
 - Learning Outcome: Apply hashing to classic problems: Two Sum (one-pass), substring anagrams, etc.
 - Topics Covered: Using hash to reduce complexity (e.g., hash map for Two Sum to $O(n)$).
 - Step-by-Step Algorithm Explanation: Use a hash map to find complement for Two Sum in one pass.
 - Time Complexity: $O(n)$ for single-pass hash solution.
 - Space Complexity: $O(n)$.

- Dry Run: Two Sum example [2,7,11,15], target=9.
- Visual Explanation: Table of complements seen so far.
- Practical Exercises: Use hash to solve "subarray sum equals k" (prefix sum hash).
- Mini Assignment: Given logs, use hash map to count frequency of events per category.
- Common Mistakes: Not checking map before inserting, causing missed complement.
- Interview Questions: "How can hash map optimize Two Sum?"
- Quiz Questions: State result of one-pass hash for a simple input.

Chapter 10: Two Pointers

- **Learning Objective:** Master the two-pointer technique to solve array/string problems in linear time **【25†L134-L142】** by moving pointers in a structured way.
- **Skills Covered:** Opposing and same-direction pointers, application examples (pair sums, palindrome checks, merge sort merging).
- **Prerequisites:** Arrays (Chapter 5), Sorted data assumptions if applicable.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 10.1: Two Pointers – Opposing Ends**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use two indices at start/end to find pairs or check conditions (e.g., sorted two-sum, palindrome).
 - Topics Covered: Setup pointers $i=0$, $j=n-1$; move based on sum or comparison.
 - Step-by-Step Algorithm Explanation: Solve "two sum sorted" by moving pointers inward if sum too large/small.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: Sum to 10 in [1,3,4,7,10] with pointers from both ends.
 - Visual Explanation: Arrows moving toward center on array.
 - Practical Exercises: Check if array is a palindrome (two-pointer equality).
 - Mini Assignment: Given sorted array, find all pairs with sum K.
 - Common Mistakes: Incorrect pointer movement order; missing cross condition.
 - Interview Questions: "Explain two-pointer technique with example."
 - Quiz Questions: For array [1,2,3,4,5], target 6, what pointer moves occur?
- **Lesson 10.2: Two Pointers – Same Direction (Sliding Window Intro)**

- Difficulty Level: Intermediate
- Learning Outcome: Use two pointers both moving forward to define a dynamic window (intro to sliding window).
- Topics Covered: Left and right pointers both start at beginning, expand/shrink window.
- Step-by-Step Algorithm Explanation: Example: find subarray with sum $\geq S$ by moving right then left pointers.
- Time Complexity: $O(n)$.
- Space Complexity: $O(1)$.
- Dry Run: Find subarray sum ≥ 7 in $[2, 1, 5, 2, 3, 2]$.
- Visual Explanation: Sliding window highlighted on array.
- Practical Exercises: Implement “shortest subarray with sum $\geq K$.”
- Mini Assignment: Longest substring with at most 2 distinct chars (sliding window variant).
- Common Mistakes: Forget to shrink window after achieving condition.
- Interview Questions: “Difference between two-pointer and sliding window?”
- Quiz Questions: After processing $[2, 1, 5, 2]$ for sum ≥ 6 , what are window bounds?

• Lesson 10.3: Additional Two Pointer Applications

- Difficulty Level: Advanced
- Learning Outcome: Explore more problems (triplets with sum, merging sorted arrays, etc.) using two-pointer logic.
- Topics Covered: 3-sum (fix one element + two-pointer for rest), merge two sorted arrays.
- Step-by-Step Algorithm Explanation: 3-sum: fix first, then solve two-sum with remaining array.
- Time Complexity: $O(n^2)$.
- Space Complexity: $O(1)$ extra.
- Dry Run: 3-sum example in $[-1, 0, 1, 2, -1, -4]$.
- Visual Explanation: Array pointers moving for each fixed element.
- Practical Exercises: Merge two sorted arrays in-place (without extra space).
- Mini Assignment: Given two sorted lists, return intersection using two pointers.
- Common Mistakes: Not skipping duplicates, off-by-one on triplet loop.
- Interview Questions: “How to find all triplets that sum to 0?”
- Quiz Questions: Steps of merging $[1, 4, 7]$ and $[2, 5, 8]$ into one sorted array.

Chapter 11: Sliding Window

- **Learning Objective:** Learn the sliding window technique for fixed and variable-size window problems on arrays/strings 【30†L46-L53】 , optimizing window computations to $O(n)$ 【30†L46-L53】 .
- **Skills Covered:** Fixed-size window (find max/min/avg in subarrays), variable window (expand/contract to meet condition).
- **Prerequisites:** Arrays (Chapter 5), Two pointers (Chapter 10).
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 3
- **Lesson 11.1: Fixed-Size Sliding Window**
 - Difficulty Level: Beginner
 - Learning Outcome: Solve problems with a given window size k in linear time.
 - Topics Covered: Maintain window sum/maximum, use deque for max in window.
 - Step-by-Step Algorithm Explanation: Compute sum of first k elements, then slide window updating sum.
 - Time Complexity: $O(n)$ for sum; $O(n)$ for max if using deque.
 - Space Complexity: $O(k)$ or $O(n)$.
 - Dry Run: Window sum of size 3 on $[2, 1, 5, 2, 3, 2]$.
 - Visual Explanation: Highlight moving window.
 - Practical Exercises: Find max of each subarray of size k .
 - Mini Assignment: Count distinct elements in each window of size k (use map).
 - Common Mistakes: Forgetting to remove old element from sum/deque.
 - Interview Questions: “How to find maximum of all subarrays of size k ?”
 - Quiz Questions: Sliding window sums for example array.
- **Lesson 11.2: Variable-Size Sliding Window**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use sliding window when the window size depends on conditions.
 - Topics Covered: Expand/shrink pointers based on condition (sum $\geq K$, number of distinct chars $\leq K$).
 - Step-by-Step Algorithm Explanation: Longest substring without repeating (increment right, shrink left on repeat).
 - Time Complexity: $O(n)$ as each pointer moves once.
 - Space Complexity: $O(n)$ for storing seen elements.
 - Dry Run: Longest substring of “eacea” (answer “eae”).
 - Visual Explanation: Growing and shrinking window on string.

- Practical Exercises: Longest subarray with sum $\leq K$.
- Mini Assignment: Minimum length subarray sum $\geq S$.
- Common Mistakes: Not handling window reset correctly, double-counting chars.
- Interview Questions: “Explain sliding window with an example.”
- Quiz Questions: Result of sliding window on sample input “abbcbc”.
- **Lesson 11.3: Sliding Window with Dynamic Constraints**
 - Difficulty Level: Advanced
 - Learning Outcome: Tackle complex window problems (dynamic K, multiple conditions).
 - Topics Covered: E.g., character replacement with one allowance, window with two pointers and counter.
 - Step-by-Step Algorithm Explanation: Replace at most k characters to make substring of repeating char (sliding window + frequency map).
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$ (alphabet count).
 - Dry Run: “AABABBA”, $k=1$, find max repeating after replacement.
 - Visual Explanation: Frequency count table changes as window moves.
 - Practical Exercises: Longest subarray where difference (max-min) $\leq k$.
 - Mini Assignment: Window for median in sliding window (advanced).
 - Common Mistakes: Not updating max frequency correctly.
 - Interview Questions: “How to solve the longest repeating character replacement problem?”
 - Quiz Questions: Window contents after 3 steps of example algorithm.

Chapter 12: Prefix Sum

- **Learning Objective:** Use prefix-sum technique to answer range-sum queries efficiently [32†L32-L39] and solve related problems.
- **Skills Covered:** Building prefix-sum array in $O(n)$ and answering sum queries in $O(1)$ [32†L32-L39] ; difference between prefix and suffix sums.
- **Prerequisites:** Arrays (Chapter 5).
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 12.1: Building Prefix Sums**
 - Difficulty Level: Beginner
 - Learning Outcome: Construct prefix-sum array for a given integer array.
 - Topics Covered: Cumulative sum concept, handling of initial index.
 - Step-by-Step Algorithm Explanation: Iteratively compute running total.

- Time Complexity: $O(n)$ preprocessing.
- Space Complexity: $O(n)$.
- Dry Run: Prefix-sums of $[2,4,6,8] = [2,6,12,20]$.
- Visual Explanation: Array of prefix values above original array.
- Practical Exercises: Given array, build and display prefix-sum array.
- Mini Assignment: Modify code to handle dynamic updates (Fenwick tree preview).
- Common Mistakes: Off-by-one indexing, forgetting to set `prefix[0]`.
- Interview Questions: “How can prefix sums speed up range queries?”
- Quiz Questions: Compute prefix sums for small input.
- **Lesson 12.2: Range Sum Queries**
 - Difficulty Level: Beginner
 - Learning Outcome: Answer any $[L,R]$ sum query in $O(1)$ time using prefix array.
 - Topics Covered: Using formula: $\text{sum}[L..R] = \text{prefix}[R] - \text{prefix}[L-1]$.
 - Step-by-Step Algorithm Explanation: Derive formula for sum of a subarray.
 - Time Complexity: $O(1)$ per query after $O(n)$ prep.
 - Space Complexity: $O(n)$.
 - Dry Run: Query $\text{sum}(1,3)$ in $[2,4,6,8]$ using prefix array.
 - Visual Explanation: Diagram showing subtraction of two prefix values.
 - Practical Exercises: Implement range sum queries on static array.
 - Mini Assignment: Adapt to 2D prefix sums for matrix queries.
 - Common Mistakes: Not handling case $L=0$ (i.e., use `prefix[R]` directly).
 - Interview Questions: “Explain how prefix sums answer queries quickly.”
 - Quiz Questions: Compute sum of subarray using prefix array.

Chapter 13: Difference Array

- **Learning Objective:** Learn the difference-array technique for efficient range updates **【34†L27-L34】** : apply multiple range addition queries in $O(1)$ each with $O(n)$ final reconstruction.
- **Skills Covered:** Range update trick, building difference array, converting diff array back to actual array with prefix sum.
- **Prerequisites:** Arrays, prefix sums (Chapter 12).
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 2
- **Lesson 13.1: Difference Array Fundamentals**
 - Difficulty Level: Intermediate

- Learning Outcome: Understand how a difference array supports $O(1)$ range updates 【34†L27-L34】 .
- Topics Covered: Creating a diff array of zeros; updating endpoints for range $[l, r]$.
- Step-by-Step Algorithm Explanation: Illustrate adding v to $\text{diff}[l]$, subtracting at $\text{diff}[r+1]$.
- Time Complexity: $O(1)$ per update; $O(n)$ to apply all updates.
- Space Complexity: $O(n)$.
- Dry Run: Apply updates {add 5 to $[1, 3]$, add -2 to $[2, 4]$ } on an initial zero array.
- Visual Explanation: Marker at l (+) and at $r+1$ (-) on diff array.
- Practical Exercises: Perform manual diff updates on small example.
- Mini Assignment: Given multiple $[l, r, \text{val}]$ operations, compute final array.
- Common Mistakes: Forgetting $\text{diff}[r+1]$ update; handling boundary $r = n-1$.
- Interview Questions: “How to add x to all elements in $[l..r]$ quickly?”
- Quiz Questions: State diff array after single range update.
- **Lesson 13.2: Applying Difference Array**
 - Difficulty Level: Intermediate
 - Learning Outcome: Convert the diff array back to final array with prefix sums; apply to solve queries.
 - Topics Covered: Computing prefix sum of diff to reconstruct original array after updates.
 - Step-by-Step Algorithm Explanation: Prefix-sum pass on diff array after all updates.
 - Time Complexity: $O(n)$ for reconstruction.
 - Space Complexity: $O(n)$.
 - Dry Run: Reconstruct final array from diff after updates in previous lesson.
 - Visual Explanation: Each element gets cumulative effect of prior ops.
 - Practical Exercises: Given initial array and updates, compute final in code.
 - Mini Assignment: Use difference array to solve “range addition on array” problem.
 - Common Mistakes: Reconstructing prefix incorrectly (forget to apply to all indices).
 - Interview Questions: “Why does prefix-summing the diff array yield the result?”
 - Quiz Questions: Given diff array $[0, 5, 0, -5, 0]$, what is the final array?

Chapter 14: Bit Manipulation

- **Learning Objective:** Use bitwise operations to solve problems involving integers and binary representations 【40†L30-L38】 . Techniques include bit masks, shifts, and bit patterns for efficiency.

- **Skills Covered:** Basic bit operations (AND, OR, XOR, NOT), bit shifts, bit tricks (powers of two, swapping), application to problems (duplicate detection, parity).
- **Prerequisites:** Chapter 2 (binary math).
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 3
- **Lesson 14.1: Bitwise Operators and Tricks**
 - Difficulty Level: Beginner
 - Learning Outcome: Perform basic bitwise operations and understand their effects.
 - Topics Covered: AND, OR, XOR, NOT; common tricks (check even/odd, power-of-two test).
 - Step-by-Step Algorithm Explanation: E.g., swapping two numbers using XOR without temp.
 - Time Complexity: $O(1)$ per operation.
 - Space Complexity: $O(1)$.
 - Dry Run: Swap $x=5$, $y=3$ using XOR sequence.
 - Visual Explanation: Bit patterns before/after operations.
 - Practical Exercises: Implement “check if integer is power of two” using bit tricks.
 - Mini Assignment: Given an integer array where every element appears twice except one, find the unique element using XOR.
 - Common Mistakes: Off-by-one in shifts; using negative numbers carelessly with shifts.
 - Interview Questions: “What does $x \& (x-1)$ do?” (turns off lowest set bit).
 - Quiz Questions: Compute bitwise AND/OR of small numbers.
- **Lesson 14.2: Advanced Bit Manipulation Problems**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve interview problems using bits (count bits, Gray code, sign detection).
 - Topics Covered: Counting set bits (Brian Kernighan’s method), generating power sets with bit masks, bit reversals.
 - Step-by-Step Algorithm Explanation: Count bits by repeatedly removing lowest set bit.
 - Time Complexity: $O(\text{\#setBits})$.
 - Space Complexity: $O(1)$.
 - Dry Run: Count bits in 13 (1101_2) using $n \& (n-1)$.
 - Visual Explanation: Removal of least significant 1 bit in sequence.
 - Practical Exercises: Implement Gray code sequence generator for n bits.

- Mini Assignment: Flip every pair of bits in a number (swap bits 0&1, 2&3, etc.).
- Common Mistakes: Infinite loop if no clear stopping condition.
- Interview Questions: “How to count bits in an integer efficiently?”
- Quiz Questions: Result of bitwise operations (e.g., ~5).
- **Lesson 14.3: Bitmasks and Bitmasks for Subsets**
 - Difficulty Level: Advanced
 - Learning Outcome: Use bitmasks to represent subsets/combinations efficiently.
 - Topics Covered: Iterating through all subsets with bit mask, DP with bitmask (for small n).
 - Step-by-Step Algorithm Explanation: Example: generate subsets of {a,b,c} by bitmask 0..7.
 - Time Complexity: $O(2^n * n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Mask=5 (101_2) yields subset {a,c}.
 - Visual Explanation: Binary counting chart.
 - Practical Exercises: Solve “subset sum” for small arrays using bit enumeration.
 - Mini Assignment: Use bitmask DP to solve Traveling Salesman for $n \leq 15$.
 - Common Mistakes: Confusing bit order (LSB vs MSB mapping to elements).
 - Interview Questions: “Explain using bitmask for subset generation.”
 - Quiz Questions: List subsets given a bitmask value.

Chapter 15: Stack

- **Learning Objective:** Learn stack data structure (LIFO) for managing execution and solving problems like parenthesis matching, postfix evaluation, etc.
- **Skills Covered:** Stack operations (push/pop/peek), use in recursion elimination, applications (expression evaluation, backtracking).
- **Prerequisites:** Recursion concept, basic data structures.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 15.1: Stack Fundamentals**
 - Difficulty Level: Beginner
 - Learning Outcome: Implement stack (array or linked list) and use for basic tasks.
 - Topics Covered: LIFO concept, typical operations, when to use a stack.
 - Step-by-Step Algorithm Explanation: N/A (conceptual).

- Time Complexity: $O(1)$ for push/pop.
- Space Complexity: $O(n)$.
- Dry Run: Push 1,2,3 and pop sequence.
- Visual Explanation: Stack diagram showing pushes.
- Practical Exercises: Use stack to reverse a list, or to track function calls in recursion.
- Mini Assignment: Implement a stack class with error handling.
- Common Mistakes: Stack overflow when full, underflow when empty.
- Interview Questions: “Where is a stack used in computers?” (e.g., call stack)
- Quiz Questions: Sequence of outputs when performing push/pop operations.
- **Lesson 15.2: Stack Applications**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve problems: check balanced parentheses, evaluate postfix (Reverse Polish) expressions.
 - Topics Covered: Parenthesis matching, stack for expression evaluation, stock span, nearest greater element.
 - Step-by-Step Algorithm Explanation: Balanced parentheses check: push on ‘(’, pop on ‘)’.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Evaluate “3 4 + 2 *” in postfix using stack.
 - Visual Explanation: Stack evolution during evaluation.
 - Practical Exercises: Write code to validate parentheses in a string.
 - Mini Assignment: Find next greater element in an array using a stack.
 - Common Mistakes: Not checking empty stack before pop.
 - Interview Questions: “How to evaluate an infix expression using stacks?”
 - Quiz Questions: Steps to evaluate a sample postfix expression.

Chapter 16: Queue

- **Learning Objective:** Understand queue data structure (FIFO) and its variants; apply queues to problems like BFS, sliding window, scheduling.
- **Skills Covered:** Queue operations (enqueue, dequeue), circular queues, priority queue (as heap).
- **Prerequisites:** Basic data structures (stacks).
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 16.1: Queue Fundamentals**

- Difficulty Level: Beginner
- Learning Outcome: Implement a queue (array or linked list) and use for level-order processes.
- Topics Covered: FIFO concept, circular buffer, queue interface.
- Step-by-Step Algorithm Explanation: N/A (conceptual).
- Time Complexity: $O(1)$ enqueue/dequeue.
- Space Complexity: $O(n)$.
- Dry Run: Enqueue 1,2,3 and dequeue sequence.
- Visual Explanation: Queue diagram with pointers.
- Practical Exercises: Simulate customer service queue with operations.
- Mini Assignment: Implement a circular queue array.
- Common Mistakes: Confusing front/rear indices, forgetting wrap-around.
- Interview Questions: “How to implement a queue using two stacks?”
- Quiz Questions: Order of elements after series of enqueues/dequeues.
- **Lesson 16.2: Queue Applications**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use queues for BFS on graphs/trees, sliding window problems, and scheduling tasks.
 - Topics Covered: Breadth-first traversal using queue, caching (LRU using queue+map).
 - Step-by-Step Algorithm Explanation: BFS on a simple graph from a start node.
 - Time Complexity: $O(V+E)$ for BFS.
 - Space Complexity: $O(V)$ for visited.
 - Dry Run: BFS on graph with 4 nodes example.
 - Visual Explanation: Queue content during BFS.
 - Practical Exercises: Perform level order traversal of binary tree.
 - Mini Assignment: Implement queue for hot potato game simulation.
 - Common Mistakes: Marking nodes visited too late; missing increments.
 - Interview Questions: “Why BFS uses a queue?”
 - Quiz Questions: Next nodes visited in BFS after given state.

Chapter 17: Linked List

- **Learning Objective:** Learn linked list structures: singly, doubly, circular; perform operations and solve interview problems.
- **Skills Covered:** Node manipulation (insertion, deletion), pointer handling, reversing lists, cycle detection (Floyd’s), merge sorted lists.
- **Prerequisites:** Pointers/references, recursion.
- **Estimated Learning Time:** 5–6 hours.

- **Number of Lessons: 5**

- **Lesson 17.1: Singly Linked List Basics**

- Difficulty Level: Beginner
- Learning Outcome: Implement singly linked list with insertions/deletions.
- Topics Covered: Node structure, head/tail pointers, traversal.
- Step-by-Step Algorithm Explanation: Insert at head/tail; delete node.
- Time Complexity: $O(1)$ insert at head, $O(n)$ insert at tail without tail pointer.
- Space Complexity: $O(n)$ nodes.
- Dry Run: Add 3, then 5 at head; delete 3.
- Visual Explanation: Diagram of nodes linked by pointers.
- Practical Exercises: Write functions to add/remove nodes.
- Mini Assignment: Implement function to find the middle node (fast/slow pointers).
- Common Mistakes: Losing track of next pointers when deleting.
- Interview Questions: “How to reverse a linked list iteratively?”
- Quiz Questions: State result of operations on example list.

- **Lesson 17.2: Linked List Operations and Pointers**

- Difficulty Level: Intermediate
- Learning Outcome: Advanced pointer operations: reversing list, merging two lists, detecting cycles.
- Topics Covered: Reverse list algorithm, Floyd’s cycle detection (tortoise-hare), merging sorted lists.
- Step-by-Step Algorithm Explanation: Reverse $[1 \rightarrow 2 \rightarrow 3]$ to $[3 \rightarrow 2 \rightarrow 1]$.
- Time Complexity: $O(n)$ for each operation.
- Space Complexity: $O(1)$.
- Dry Run: Floyd’s algorithm on list with cycle.
- Visual Explanation: Two pointers moving at different speeds.
- Practical Exercises: Implement iterative and recursive reverse.
- Mini Assignment: Given two sorted lists, merge them into one sorted list.
- Common Mistakes: Mishandling of null pointers in reversing, infinite loops in cycle check.
- Interview Questions: “How do you detect and remove a cycle in a linked list?”
- Quiz Questions: Steps in reversing a given small list.

- **Lesson 17.3: Doubly and Circular Linked Lists**

- Difficulty Level: Intermediate

- Learning Outcome: Work with doubly and circular lists; insert/delete in a circular buffer.
- Topics Covered: Node with two pointers, circular list where tail links to head.
- Step-by-Step Algorithm Explanation: Insert a node in a circular list.
- Time Complexity: $O(1)$ for insert at known position.
- Space Complexity: $O(n)$.
- Dry Run: Create circular list $1 \rightarrow 2 \rightarrow 3 \rightarrow (\text{back to } 1)$.
- Visual Explanation: Diagram showing prev/next links and circular connection.
- Practical Exercises: Convert singly to doubly, or to circular.
- Mini Assignment: Solve Josephus problem using circular list simulation.
- Common Mistakes: Infinite loops not detected in circular structures.
- Interview Questions: “When use a circular list?”
- Quiz Questions: Identify next node after several steps in circular list.
- **Lesson 17.4: Linked List Examples**
 - Difficulty Level: Advanced
 - Learning Outcome: Solve classic interview problems: remove Nth from end, add numbers represented by lists.
 - Topics Covered: Two-pointer technique on lists, managing carries in sum of lists.
 - Step-by-Step Algorithm Explanation: Find nth from end by first finding length or using two pointers.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: Remove 2nd from end in $[1 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 5]$.
 - Visual Explanation: Two-pointer offset strategy in list.
 - Practical Exercises: Add two numbers given as linked lists of digits.
 - Mini Assignment: Partition linked list around a value x.
 - Common Mistakes: Off-by-one in pointer movement, forgetting to remove all nodes.
 - Interview Questions: “How to add two numbers represented by linked lists?”
 - Quiz Questions: Result list after an addition example.
- **Lesson 17.5: Linked List Practice**
 - Difficulty Level: Advanced
 - Learning Outcome: Practice with challenging problems (flatten multilevel list, copy with random pointer).
 - Topics Covered: Complex list manipulations (multiple pointers per node).
 - Step-by-Step Algorithm Explanation: Copy list with random pointers: use interleaving method.

- Time Complexity: $O(n)$.
- Space Complexity: $O(1)$ extra.
- Dry Run: Duplicate a small list with random links.
- Visual Explanation: Showing original and copy interleaved.
- Practical Exercises: Flatten a multilevel doubly linked list.
- Mini Assignment: Find intersection of two singly linked lists.
- Common Mistakes: Not resetting pointers correctly when copying.
- Interview Questions: “How to deep copy a linked list with random pointers?”
- Quiz Questions: Pointers state after an intermediate step in random-list copy.

Chapter 18: Trees

- **Learning Objective:** Learn tree data structures (binary trees) for hierarchical data; perform traversals and understand tree properties.
- **Skills Covered:** Tree traversal (preorder, inorder, postorder, level order), computing height/size, depth-first recursion on trees, basic tree problems (max depth, path sums).
- **Prerequisites:** Linked Lists, recursion.
- **Estimated Learning Time:** 5–6 hours.
- **Number of Lessons:** 5
- **Lesson 18.1: Tree Fundamentals**
 - Difficulty Level: Beginner
 - Learning Outcome: Understand tree concepts: nodes, children, root, leaves, depth.
 - Topics Covered: Terminology, binary vs general trees, representations (pointers vs array).
 - Step-by-Step Algorithm Explanation: N/A (conceptual).
 - Time Complexity: N/A (concepts).
 - Space Complexity: $O(n)$ for storing nodes.
 - Dry Run: N/A.
 - Visual Explanation: Diagram of a simple binary tree.
 - Practical Exercises: Define a tree node class and create a small tree.
 - Mini Assignment: Compute number of leaf nodes in a simple tree example.
 - Common Mistakes: Confusing depth vs height, left vs right.
 - Interview Questions: “What is a binary tree?”
 - Quiz Questions: Terms definition (parent, sibling, etc.).
- **Lesson 18.2: Binary Tree Traversals**
 - Difficulty Level: Beginner

- Learning Outcome: Perform inorder, preorder, postorder and level-order traversals.
- Topics Covered: Recursion for DFS traversals, queue for BFS (level-order).
- Step-by-Step Algorithm Explanation: Traverse example tree [1, null, 2, 3] in inorder.
- Time Complexity: $O(n)$ for all traversals.
- Space Complexity: $O(n)$ worst-case (stack/queue).
- Dry Run: Inorder and level-order on sample tree.
- Visual Explanation: Trace arrows visiting nodes in order.
- Practical Exercises: Code each traversal; print paths.
- Mini Assignment: Given inorder and preorder lists, reconstruct the tree.
- Common Mistakes: Mixing traversal orders; forgetting right subtree.
- Interview Questions: “Difference between inorder and preorder?”
- Quiz Questions: Output of given tree traversal.
- **Lesson 18.3: Binary Tree Applications**
 - Difficulty Level: Intermediate
 - Learning Outcome: Compute tree metrics: height, size, diameter; solve simple tree problems.
 - Topics Covered: Height calculation, count nodes, check balanced tree.
 - Step-by-Step Algorithm Explanation: Height of node = $\max(\text{height}(\text{left}), \text{height}(\text{right})) + 1$.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$ recursion stack.
 - Dry Run: Height of small tree.
 - Visual Explanation: Annotated tree with heights.
 - Practical Exercises: Check if tree is symmetric (mirror).
 - Mini Assignment: Compute diameter (longest path) of tree.
 - Common Mistakes: Off-by-one in height calculation.
 - Interview Questions: “How to determine if two binary trees are identical?”
 - Quiz Questions: Height of given tree.
- **Lesson 18.4: Binary Tree Problems**
 - Difficulty Level: Advanced
 - Learning Outcome: Tackle classic problems: lowest common ancestor, path sum, subtree checking.
 - Topics Covered: LCA in binary tree (not BST), root-to-leaf path sums, subtree check via traversal.
 - Step-by-Step Algorithm Explanation: Find LCA by DFS from root.
 - Time Complexity: $O(n)$ for LCA, $O(n*m)$ naive subtree check.
 - Space Complexity: $O(n)$ stack.

- Dry Run: LCA of nodes 4 and 5 in sample tree.
- Visual Explanation: Mark paths to each node and intersection.
- Practical Exercises: Determine if one tree is subtree of another.
- Mini Assignment: Path sum equals target (boolean) for any root-to-leaf.
- Common Mistakes: Not covering null children in recursion.
- Interview Questions: “How to find LCA in a binary tree?”
- Quiz Questions: Trace LCA algorithm on small tree.
- **Lesson 18.5: Tree Practice and Patterns**
 - Difficulty Level: Advanced
 - Learning Outcome: Solve complex tree tasks (flatten to linked list, serialize/deserialize).
 - Topics Covered: Tree-to-List flatten, trie-preview.
 - Step-by-Step Algorithm Explanation: Convert tree to in-order linked list (modifying pointers).
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Flatten [1,2,5,3,4,null,6] to [1,null,2,null,3,null,4,null,5,null,6].
 - Visual Explanation: Linked list outline for flatten.
 - Practical Exercises: Serialize a binary tree to a string.
 - Mini Assignment: Check if binary tree is height-balanced.
 - Common Mistakes: Lost children when flattening.
 - Interview Questions: “Explain flattening a binary tree into a linked list.”
 - Quiz Questions: Result sequence after flattening example tree.

Chapter 19: Binary Search Tree

- **Learning Objective:** Learn BST properties: $\text{left} < \text{root} < \text{right}$; implement insert/search/delete; perform traversal in sorted order.
- **Skills Covered:** BST operations, leveraging BST for fast lookup, understanding self-balancing (conceptual).
- **Prerequisites:** Binary Trees (Chapter 18).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 19.1: BST Structure and Search**
 - Difficulty Level: Beginner
 - Learning Outcome: Insert and search in BST; leverage sorted nature for $O(h)$ operations.
 - Topics Covered: BST property, recursive/iterative insert and find.
 - Step-by-Step Algorithm Explanation: Insert 7 into BST [5,3,8].

- Time Complexity: $O(h)$ (height), average $O(\log n)$.
- Space Complexity: $O(h)$ recursion, $O(1)$ iterative.
- Dry Run: Search 4 in BST [5,3,8,2,4].
- Visual Explanation: BST with left/right pointers.
- Practical Exercises: Write recursive BST insert and search functions.
- Mini Assignment: Check if a binary tree is a valid BST.
- Common Mistakes: Not handling equal keys (decide convention).
- Interview Questions: “How to search in a BST?”
- Quiz Questions: After inserting sequence, what is inorder output?
- **Lesson 19.2: BST Deletion and Traversal**
 - Difficulty Level: Intermediate
 - Learning Outcome: Implement delete operation (0,1,2 child cases) and inorder traversal.
 - Topics Covered: Deleting leaf, node with one child, two children (successor approach).
 - Step-by-Step Algorithm Explanation: Delete node 6 (with two children) example.
 - Time Complexity: $O(h)$.
 - Space Complexity: $O(h)$ recursion.
 - Dry Run: Delete 3 from [5,3,8,2,4].
 - Visual Explanation: BST before/after deletion.
 - Practical Exercises: Perform inorder traversal to verify BST property.
 - Mini Assignment: Given inorder and preorder, rebuild the BST.
 - Common Mistakes: Mishandling successor/predecessor links.
 - Interview Questions: “Explain deletion in BST.”
 - Quiz Questions: Result of deleting node with two children in sample BST.
- **Lesson 19.3: BST Applications**
 - Difficulty Level: Advanced
 - Learning Outcome: Solve BST problems: kth smallest, closest value, floor/ceil in BST.
 - Topics Covered: Inorder count, recursive search for bounds.
 - Step-by-Step Algorithm Explanation: Find 3rd smallest by in-order traversal.
 - Time Complexity: $O(h + k)$.
 - Space Complexity: $O(h)$ stack.
 - Dry Run: Closest value to 17 in BST [10,5,15,18] is 15.
 - Visual Explanation: BST with tracked best candidate.
 - Practical Exercises: Find LCA in BST (simpler than binary tree).
 - Mini Assignment: Floor and ceil values in a BST for a target.

- Common Mistakes: Not updating candidate on each step.
- Interview Questions: “How to find kth largest in a BST?”
- Quiz Questions: Inorder sequence to find kth element.

Chapter 20: Heap

- **Learning Objective:** Understand heap data structure (binary heap) as priority queue; use heap for priority-based problems.
- **Skills Covered:** Min-heap and max-heap operations (insert, extract), building heap, heap-based algorithms (heap sort preview).
- **Prerequisites:** Binary trees (heap is a tree), arrays.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 20.1: Heap Concepts**
 - Difficulty Level: Beginner
 - Learning Outcome: Learn min-heap property and how to implement with arrays.
 - Topics Covered: Binary heap shape and heap property, array representation.
 - Step-by-Step Algorithm Explanation: Insert into heap (heapify-up).
 - Time Complexity: $O(\log n)$ per operation.
 - Space Complexity: $O(n)$.
 - Dry Run: Insert 5 into heap [1,3,8,7,2] (as array).
 - Visual Explanation: Array vs tree view of heap.
 - Practical Exercises: Build a heap by successive inserts.
 - Mini Assignment: Convert array into heap using “heapify” process.
 - Common Mistakes: Not swapping correctly up/down heap.
 - Interview Questions: “Why is heap insertion $O(\log n)$?”
 - Quiz Questions: Parent index of element at position 5 (0-index).
- **Lesson 20.2: Heap Operations and PQ**
 - Difficulty Level: Intermediate
 - Learning Outcome: Implement extract-max/min and peek, and use heap as priority queue.
 - Topics Covered: Remove root (heapify-down), peek top priority, implementing a max-heap or min-heap.
 - Step-by-Step Algorithm Explanation: Remove root and reheapify [1,3,8,7,2].
 - Time Complexity: $O(\log n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: Extract min from heap [1,2,5,7,3].
 - Visual Explanation: Heap before/after extract.

- Practical Exercises: Use priority queue to merge k sorted lists (small example).
- Mini Assignment: Implement custom comparator for priority queue in your language.
- Common Mistakes: Off-by-one on child calculations; not percolating correctly.
- Interview Questions: “How to find k largest elements using a heap?”
- Quiz Questions: New root after extracting from a given heap.

Chapter 21: Priority Queue

- **Learning Objective:** Use priority queue abstractions (often implemented with a heap) for scheduling tasks, processing in priority order, and other greedy algorithms.
- **Skills Covered:** Language-specific PQ usage; designing comparators; real-world uses (event simulation, Dijkstra).
- **Prerequisites:** Heap (Chapter 20).
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 21.1: Priority Queue Basics**
 - Difficulty Level: Beginner
 - Learning Outcome: Use built-in priority queue in code or implement one; understand default min/max behavior.
 - Topics Covered: Using `std::priority_queue` (C++), `heapq` (Python), or custom heap class.
 - Step-by-Step Algorithm Explanation: N/A (library usage).
 - Time Complexity: $O(\log n)$ per enqueue/dequeue.
 - Space Complexity: $O(n)$.
 - Dry Run: Add tasks with priorities; show queue ordering.
 - Visual Explanation: Priority queue as heap.
 - Practical Exercises: Schedule jobs by priority.
 - Mini Assignment: Create task scheduler where each job has a deadline and duration.
 - Common Mistakes: Not resetting queue properly between tasks.
 - Interview Questions: “What’s difference between queue and priority queue?”
 - Quiz Questions: Order of processing tasks by given priorities.
- **Lesson 21.2: Priority Queue Applications**
 - Difficulty Level: Intermediate

- Learning Outcome: Solve problems like merging sorted data streams, top-k elements using PQ.
- Topics Covered: Merge k sorted arrays (k-way merge), running medians.
- Step-by-Step Algorithm Explanation: Maintain min-heap of k elements for streaming merge.
- Time Complexity: $O(n \log k)$.
- Space Complexity: $O(k)$.
- Dry Run: Merge [1,4,7] and [2,5,6] by priority queue.
- Visual Explanation: Heap showing next elements from each list.
- Practical Exercises: Find top 3 frequent words in a list using PQ.
- Mini Assignment: Simulate median maintenance with two heaps (max-heap, min-heap).
- Common Mistakes: Off-by-one in indexing list heads.
- Interview Questions: “How to find k smallest numbers in a large stream?”
- Quiz Questions: Next popped element from a PQ with given contents.

Chapter 22: Trie

- **Learning Objective:** Understand trie (prefix tree) data structure for fast prefix-based searches. Cover insertion, search, and prefix queries.
- **Skills Covered:** Implement trie with nodes (usually alphabet children); use for word games, autocomplete, prefix matching.
- **Prerequisites:** Trees (Chapter 18), strings (Chapter 6).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 22.1: Trie Fundamentals**
 - Difficulty Level: Intermediate
 - Learning Outcome: Implement a basic trie to store a set of strings.
 - Topics Covered: Trie node structure (array or map of children), root node concept.
 - Step-by-Step Algorithm Explanation: Insert “apple” character by character, marking end.
 - Time Complexity: $O(m)$ per insert/search (m =length of word).
 - Space Complexity: $O(\text{total chars})$.
 - Dry Run: Insert words “ant”, “ball” into empty trie.
 - Visual Explanation: Tree with branches for each character.
 - Practical Exercises: Implement insert and search in a trie.
 - Mini Assignment: Given a list of words, build trie and find all starting with prefix “pre”.
 - Common Mistakes: Not creating new nodes properly.

- Interview Questions: “Why use trie over hash map for prefix search?”
- Quiz Questions: Depth of nodes representing given words.
- **Lesson 22.2: Trie Search and Prefix**
 - Difficulty Level: Intermediate
 - Learning Outcome: Perform prefix search and delete words from trie.
 - Topics Covered: Searching full word, checking if any word has given prefix, deletion cleanup.
 - Step-by-Step Algorithm Explanation: Search “test” by traversing nodes; search prefix “te”.
 - Time Complexity: $O(m)$.
 - Space Complexity: $O(1)$ extra.
 - Dry Run: Search for “app” in trie with “apple”.
 - Visual Explanation: Node marking if prefix.
 - Practical Exercises: Count words with a given prefix using trie.
 - Mini Assignment: Implement word deletion from trie.
 - Common Mistakes: Forgetting to mark end-of-word or handle prefix termination.
 - Interview Questions: “How to find if any word in dict starts with prefix?”
 - Quiz Questions: Checking prefix existence in sample trie.
- **Lesson 22.3: Advanced Trie Uses**
 - Difficulty Level: Advanced
 - Learning Outcome: Use trie for problems like word search on board, multi-word autocompletion.
 - Topics Covered: Trie + DFS/BFS for board search (Word Search II), storing frequency.
 - Step-by-Step Algorithm Explanation: Word search II example: backtracking using trie for pruning.
 - Time Complexity: Depends on board size and dictionary size.
 - Space Complexity: $O(\text{total nodes in trie})$.
 - Dry Run: Find words in 2×2 board using example trie.
 - Visual Explanation: Board with trie-driven search paths.
 - Practical Exercises: Given keyboard dictionary, implement autocomplete suggestions.
 - Mini Assignment: Design contact search by phone keypad using trie.
 - Common Mistakes: Not marking end-of-word in trie for search.
 - Interview Questions: “Why is trie efficient for prefix searches?”
 - Quiz Questions: Path in trie for a given word.

Chapter 23: Graphs

- **Learning Objective:** Learn graph data structures and traversal algorithms (BFS, DFS) 【12†L123-L130】 . Solve connectivity and shortest path problems.
- **Skills Covered:** Graph representation (adj list/matrix), BFS, DFS, topological sort, shortest path (Dijkstra, Bellman-Ford, Floyd-Warshall), MST (Prim, Kruskal), Union-Find (DSU).
- **Prerequisites:** Trees, queues (Chapter 16).
- **Estimated Learning Time:** 6–8 hours.
- **Number of Lessons:** 6
- **Lesson 23.1: Graph Basics and BFS**
 - Difficulty Level: Beginner
 - Learning Outcome: Represent graphs (directed/undirected) and traverse using BFS (level order).
 - Topics Covered: Adjacency list/matrix, queue-based BFS, visited array.
 - Step-by-Step Algorithm Explanation: BFS from node 1 on a sample graph.
 - Time Complexity: $O(V+E)$.
 - Space Complexity: $O(V)$.
 - Dry Run: BFS on graph $1-\{2,3\}$, $2-\{4\}$.
 - Visual Explanation: Layers of nodes by distance.
 - Practical Exercises: Determine distance (shortest path length) from start in unweighted graph.
 - Mini Assignment: Check if graph is bipartite using BFS.
 - Common Mistakes: Forgetting to mark visited on enqueue vs dequeue.
 - Interview Questions: “How to implement BFS on a graph?”
 - Quiz Questions: BFS order of small graph.
- **Lesson 23.2: DFS and Topological Sort**
 - Difficulty Level: Intermediate
 - Learning Outcome: Traverse graphs depth-first; detect cycles; perform topological sort on DAG.
 - Topics Covered: Recursive DFS, use stack/time for topo order.
 - Step-by-Step Algorithm Explanation: DFS from node 1 on sample graph.
 - Time Complexity: $O(V+E)$.
 - Space Complexity: $O(V)$ recursion.
 - Dry Run: DFS on graph $1 \rightarrow 2 \rightarrow 3$, record finish times.
 - Visual Explanation: Recursion stack with backtracking.
 - Practical Exercises: Topologically sort a DAG (course scheduling problem).
 - Mini Assignment: Detect cycle in directed graph using DFS.

- Common Mistakes: Not tracking recursion stack for directed cycle.
- Interview Questions: “What is topological sort and when is it possible?”
- Quiz Questions: Topo order of given DAG.
- **Lesson 23.3: Shortest Path – Dijkstra**
 - Difficulty Level: Advanced
 - Learning Outcome: Compute shortest paths from source in weighted graph with non-negative weights.
 - Topics Covered: Priority queue (min-heap) use, relaxation of edges.
 - Step-by-Step Algorithm Explanation: Dijkstra on small weighted graph.
 - Time Complexity: $O((V+E) \log V)$.
 - Space Complexity: $O(V)$ for dist array.
 - Dry Run: Update distances stepwise.
 - Visual Explanation: Table of current distances updating.
 - Practical Exercises: Implement Dijkstra using adjacency list.
 - Mini Assignment: Modify to find shortest path tree.
 - Common Mistakes: Not updating priority queue properly on relax.
 - Interview Questions: “Why can’t Dijkstra handle negative weights?”
 - Quiz Questions: Distance updates for given next node pick.
- **Lesson 23.4: Shortest Path – Bellman-Ford & Floyd-Warshall**
 - Difficulty Level: Advanced
 - Learning Outcome: Handle graphs with negative weights (Bellman-Ford) and all-pairs shortest paths (Floyd-Warshall).
 - Topics Covered: Bellman-Ford relaxation $V-1$ times, detect negative cycle; Floyd-Warshall DP.
 - Step-by-Step Algorithm Explanation: Bellman-Ford example on 4-node graph.
 - Time Complexity: $O(VE)$ Bellman, $O(V^3)$ Floyd.
 - Space Complexity: $O(V)$ for Bellman, $O(V^2)$ for Floyd.
 - Dry Run: Relax edges of small graph iteratively.
 - Visual Explanation: Table of distances updated per iteration.
 - Practical Exercises: Apply Floyd-Warshall to small matrix.
 - Mini Assignment: Detect negative cycle with Bellman-Ford.
 - Common Mistakes: Off-by-one in iterations, forgetting initial distances.
 - Interview Questions: “How to detect negative cycle in graph?”
 - Quiz Questions: Next distance updates in Bellman-Ford iteration.
- **Lesson 23.5: Minimum Spanning Tree (Prim, Kruskal)**
 - Difficulty Level: Intermediate

- Learning Outcome: Find MST of weighted graph: Prim's (with PQ) and Kruskal's (with DSU).
- Topics Covered: Greedy selection of edges, union-find structure.
- Step-by-Step Algorithm Explanation: Kruskal example on small graph with edge weights.
- Time Complexity: $O(E \log E)$ (sorting edges).
- Space Complexity: $O(E+V)$.
- Dry Run: Add edges in increasing order, skip cycles.
- Visual Explanation: Graph highlighting chosen MST edges.
- Practical Exercises: Code union-find (DSU) with path compression.
- Mini Assignment: Compare edge counts in MST vs original.
- Common Mistakes: Forgetting to skip cycles in Kruskal.
- Interview Questions: "Difference between Prim's and Kruskal's algorithms?"
- Quiz Questions: Edges chosen in a given MST example.
- **Lesson 23.6: Union-Find (DSU)**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use disjoint-set union for connectivity: union and find operations.
 - Topics Covered: Union by rank/size, path compression optimizations.
 - Step-by-Step Algorithm Explanation: Union sets of elements step by step.
 - Time Complexity: Amortized $\sim O(\alpha(n))$ (almost constant).
 - Space Complexity: $O(n)$.
 - Dry Run: Union 1&2, 2&3; find(3).
 - Visual Explanation: Forest of trees for sets.
 - Practical Exercises: Implement DSU and use it to count components.
 - Mini Assignment: Use DSU for Kruskal's MST in code.
 - Common Mistakes: Not updating parent pointers; not using rank can degrade performance.
 - Interview Questions: "What is path compression?"
 - Quiz Questions: State parent of nodes after union operations.

Chapter 24: Greedy Algorithms

- **Learning Objective:** Understand greedy paradigm for optimization: always pick the "best immediate" choice and move forward **【42†L32-L34】**. Recognize when greedy works (matroids) and fails (compare to DP).
- **Skills Covered:** Greedy solutions to classic problems (activity selection, coin change, scheduling, Huffman).
- **Prerequisites:** Sorting (Chapter 8), PQ (Chapter 21).
- **Estimated Learning Time:** 4–5 hours.

- **Number of Lessons: 4**
- **Lesson 24.1: Greedy Paradigm and Examples**
 - Difficulty Level: Intermediate
 - Learning Outcome: Implement greedy solutions; know greedy-choice property.
 - Topics Covered: Activity selection, interval scheduling, coin change (canonical systems).
 - Step-by-Step Algorithm Explanation: Select non-overlapping intervals by earliest finish time.
 - Time Complexity: $O(n \log n)$ due to sorting of intervals.
 - Space Complexity: $O(n)$.
 - Dry Run: Pick intervals in example set.
 - Visual Explanation: Timeline marking selected tasks.
 - Practical Exercises: Implement activity selection.
 - Mini Assignment: Change coin denominations to a non-canonical set and show greedy fails.
 - Common Mistakes: Choosing wrong sort key (start vs end time).
 - Interview Questions: “Why does greedy work for activity selection?”
 - Quiz Questions: List selected intervals from sorted list.
- **Lesson 24.2: Huffman Coding and Scheduling**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve Huffman coding (minimize weighted path length) and minimum platforms problem (scheduling).
 - Topics Covered: Huffman tree construction; scheduling trains at platforms by arrival/departure times.
 - Step-by-Step Algorithm Explanation: Build Huffman tree by combining lowest frequencies.
 - Time Complexity: $O(n \log n)$ using PQ.
 - Space Complexity: $O(n)$.
 - Dry Run: Huffman on frequencies [5,9,12,13,16,45].
 - Visual Explanation: Huffman tree with binary codes.
 - Practical Exercises: Implement Huffman coding using min-heap.
 - Mini Assignment: Given arrival/departure arrays, find minimum platforms.
 - Common Mistakes: Not sorting events or proper PQ usage.
 - Interview Questions: “Explain how greedy builds Huffman tree.”
 - Quiz Questions: Height of Huffman tree for example.
- **Lesson 24.3: Greedy with Data Structures**
 - Difficulty Level: Advanced

- Learning Outcome: Use greedy in combination with DS (PQ, sorting) for complex tasks (connecting ropes, minimizing cash flow).
- Topics Covered: Connect ropes with minimum cost, scheduling with deadlines (job sequencing).
- Step-by-Step Algorithm Explanation: Always combine two smallest ropes first.
- Time Complexity: $O(n \log n)$.
- Space Complexity: $O(n)$.
- Dry Run: Merge ropes [4,3,2,6] stepwise.
- Visual Explanation: Sequence of merges in heap.
- Practical Exercises: Given deadlines and profits, schedule to maximize profit (greedy by profit).
- Mini Assignment: Activity selection variant with weights (greedy vs DP discussion).
- Common Mistakes: Not using PQ leads to higher cost.
- Interview Questions: “How to minimize cost of connecting ropes?”
- Quiz Questions: Cost calculation steps for given rope lengths.
- **Lesson 24.4: Greedy Theory vs DP**
 - Difficulty Level: Advanced
 - Learning Outcome: Understand when greedy fails (lack of optimal substructure) and DP is needed 【42†L65-L74】 .
 - Topics Covered: Compare greedy vs DP via coin change example; discuss greedy-choice property necessity.
 - Step-by-Step Algorithm Explanation: Counterexample for greedy coin change.
 - Time Complexity: $O(n)$ for greedy; $O(n \cdot W)$ for coin DP.
 - Space Complexity: $O(n)$ vs $O(n \cdot W)$.
 - Dry Run: Show greedy picks [4,1,1] vs optimal [3,3] for amount 6 with coins {1,3,4}.
 - Visual Explanation: Graph of greedy vs optimal.
 - Practical Exercises: Implement DP solution for coin change and compare with greedy on test cases.
 - Mini Assignment: Find a greedy vs DP problem pair (e.g., fraction knapsack vs 0/1 knapsack).
 - Common Mistakes: Assuming greedy always works.
 - Interview Questions: “Give an example where greedy fails.”
 - Quiz Questions: Determine optimal vs greedy coins for amount with given denominations.

Chapter 25: Dynamic Programming (DP)

- **Learning Objective:** Master DP by memoization and tabulation: solve overlapping subproblems with optimal substructure. Cover classic DP problems on arrays, strings, trees, graphs, bitmask DP.
- **Skills Covered:** Recognize DP patterns (knapsack, LIS, LCS, matrix chain, bitmask state DP), implement memo/table, optimize state space.
- **Prerequisites:** Recursion, greedy concepts, arrays/strings (Chapters 5-6).
- **Estimated Learning Time:** 7–9 hours.
- **Number of Lessons:** 6
- **Lesson 25.1: DP Fundamentals (Memoization vs Tabulation)**
 - Difficulty Level: Intermediate
 - Learning Outcome: Differentiate top-down vs bottom-up; convert naive recursive to DP.
 - Topics Covered: Overlapping subproblems, caching, base cases.
 - Step-by-Step Algorithm Explanation: Fibonacci with recursion vs memo.
 - Time Complexity: $O(n)$ vs $O(2^n)$.
 - Space Complexity: $O(n)$ (call stack+cache).
 - Dry Run: Compute fib(5) with recursion vs memo.
 - Visual Explanation: DP table filling vs recursion tree pruned.
 - Practical Exercises: Memoize factorial (not needed but practice), fib with DP.
 - Mini Assignment: Convert recursive LCS to tabulation.
 - Common Mistakes: Wrong subproblem state definition.
 - Interview Questions: “Memoization vs Tabulation difference?”
 - Quiz Questions: Identify repeated calls in naive recursion.
- **Lesson 25.2: Classic DP Problems (Knapsack, LIS, LCS)**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve 0/1 knapsack, longest increasing subsequence ($n \log n$), longest common subsequence.
 - Topics Covered: DP table for knapsack, patience sorting (LIS), 2D DP for LCS.
 - Step-by-Step Algorithm Explanation: LCS dp table calculation.
 - Time Complexity: $O(nW)$, $O(n \log n)$ LIS, $O(mn)$ LCS.
 - Space Complexity: $O(nW)$ for knapsack, $O(mn)$ for LCS.
 - Dry Run: Fill LCS table for “abc” vs “ac”.
 - Visual Explanation: DP table matrix annotated.
 - Practical Exercises: Implement knapsack DP with integer weights.
 - Mini Assignment: Find all LIS, not just length.
 - Common Mistakes: Off-by-one on indices in 2D DP.

- Interview Questions: “Difference between LCS and subsequence problems.”
- Quiz Questions: LCS table cell values for sample inputs.
- **Lesson 25.3: DP on Strings (Edit Distance, Regex Matching)**
 - Difficulty Level: Advanced
 - Learning Outcome: Use DP for string-edit problems: Levenshtein distance, wildcard/regex match.
 - Topics Covered: Edit distance DP, wildcard '*' handling, dynamic 2D DP grids.
 - Step-by-Step Algorithm Explanation: Fill matrix for “kitten” vs “sitting”.
 - Time Complexity: $O(n*m)$.
 - Space Complexity: $O(n*m)$.
 - Dry Run: Edit distance of “abc”→“adc”.
 - Visual Explanation: Grid of comparisons.
 - Practical Exercises: Implement wildcard matching ('?' and '*').
 - Mini Assignment: Simplified regex DP for patterns a-z and ‘.’.
 - Common Mistakes: Not handling base cases of empty string.
 - Interview Questions: “How to compute edit distance?”
 - Quiz Questions: Small DP grid step for edit.
- **Lesson 25.4: DP on Trees**
 - Difficulty Level: Advanced
 - Learning Outcome: Solve DP problems defined on tree structure (tree DP).
 - Topics Covered: Examples: max sum of non-adjacent nodes, tree path DP.
 - Step-by-Step Algorithm Explanation: Max-sum on tree: include/exclude root.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Compute max-sum on small tree.
 - Visual Explanation: Tree with include/exclude labels.
 - Practical Exercises: “House Robber III” problem (tree).
 - Mini Assignment: Diameter or longest path in tree DP.
 - Common Mistakes: Overlapping subtrees, forgetting to consider children choice.
 - Interview Questions: “How to do DP on trees?”
 - Quiz Questions: DP values for a node given children results.
- **Lesson 25.5: DP on Graphs (Bitmask)**
 - Difficulty Level: Advanced
 - Learning Outcome: Tackle DP on graphs or sets of nodes using bitmask (e.g. Hamiltonian path, TSP).
 - Topics Covered: State = (mask, position), transitions.

- Step-by-Step Algorithm Explanation: Bitmask TSP DP recurrence.
- Time Complexity: $O(n^2 * 2^n)$.
- Space Complexity: $O(n * 2^n)$.
- Dry Run: Hamiltonian path on 3 nodes with mask.
- Visual Explanation: DP table indexed by mask.
- Practical Exercises: Solve “bitmask DP for TSP” on small graph.
- Mini Assignment: DP for subset sum vs 0/1 knapsack equivalence.
- Common Mistakes: Mask representation, bit operations.
- Interview Questions: “What is bitmask DP?”
- Quiz Questions: Next state for given mask in DP step.
- **Lesson 25.6: Advanced DP Patterns**
 - Difficulty Level: Advanced
 - Learning Outcome: Recognize patterns (knapsack, grid, tree, sequence) and optimize DP (space reduction).
 - Topics Covered: Rolling array optimization, critical transitions.
 - Step-by-Step Algorithm Explanation: Space-optimized knapsack 1D.
 - Time Complexity: Same as unoptimized.
 - Space Complexity: $O(n)$ vs $O(n*W)$.
 - Dry Run: Fill 1D DP for knapsack using backward iteration.
 - Visual Explanation: 2D→1D DP mapping.
 - Practical Exercises: Convert 2D DP (LCS) to 1D.
 - Mini Assignment: Identify if a problem can be reduced to a knapsack DP form.
 - Common Mistakes: Overwriting needed values in 1D incorrectly.
 - Interview Questions: “How to optimize space in DP?”
 - Quiz Questions: Last row of DP vs final 1D array.

Chapter 26: Segment Tree

- **Learning Objective:** Learn segment tree for range queries/updates: build tree in $O(n)$, answer queries/point updates in $O(\log n)$.
- **Skills Covered:** Tree representation in array, range sum/min queries, lazy propagation basics (if time).
- **Prerequisites:** Trees (Chapter 18), arrays.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 2
- **Lesson 26.1: Building Segment Tree**
 - Difficulty Level: Advanced
 - Learning Outcome: Construct a segment tree for range sum or min.

- Topics Covered: Recursively split array into halves, store sums.
- Step-by-Step Algorithm Explanation: Build tree for [1,3,5,7] (range sums).
- Time Complexity: $O(n)$.
- Space Complexity: $O(4n) \sim O(n)$.
- Dry Run: Show tree nodes for sample range.
- Visual Explanation: Binary tree with segment ranges.
- Practical Exercises: Implement build function of segtree.
- Mini Assignment: Query sum from index 1 to 3.
- Common Mistakes: Indexing child nodes incorrectly.
- Interview Questions: “How is segment tree different from prefix sums?”
- Quiz Questions: Tree node values after building.
- **Lesson 26.2: Queries and Updates (Lazy Propagation)**
 - Difficulty Level: Advanced
 - Learning Outcome: Answer range queries and perform updates efficiently; understand lazy updates.
 - Topics Covered: Query logic, point update, range update (lazy).
 - Step-by-Step Algorithm Explanation: Sum query [1,3] example.
 - Time Complexity: $O(\log n)$ per query/update ($O(\log n)$ with lazy).
 - Space Complexity: $O(n)$.
 - Dry Run: Update index 2 to 10, then query again.
 - Visual Explanation: Highlighted path from root to leaf for query.
 - Practical Exercises: Implement range increment with lazy flags.
 - Mini Assignment: Use segtree for range max query.
 - Common Mistakes: Not propagating lazy to children.
 - Interview Questions: “What is lazy propagation?”
 - Quiz Questions: Result of query after one update.

Chapter 27: Fenwick Tree (Binary Indexed Tree)

- **Learning Objective:** Learn Fenwick Tree as an alternative to segment tree for prefix sums and updates in $O(\log n)$.
- **Skills Covered:** Build BIT from array; query prefix sum; update single element.
- **Prerequisites:** Prefix sums, bit manipulation.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 27.1: Fenwick Tree Basics**
 - Difficulty Level: Intermediate
 - Learning Outcome: Construct Fenwick tree from array; understand lowbit property.

- Topics Covered: BIT array, using i & $-i$ to traverse parents.
- Step-by-Step Algorithm Explanation: Build by adding each element.
- Time Complexity: $O(n \log n)$ build (or $O(n)$ optimized), $O(\log n)$ operations.
- Space Complexity: $O(n)$.
- Dry Run: Construct BIT for $[1, 2, 3, 4]$.
- Visual Explanation: BIT array layout with cumulative ranges.
- Practical Exercises: Implement $\text{add}(i, \text{val})$ and $\text{query}(i)$.
- Mini Assignment: Use BIT to compute prefix sums.
- Common Mistakes: 1-index vs 0-index confusion.
- Interview Questions: “How does Fenwick tree compare to segment tree?”
- Quiz Questions: BIT array content after inserts.
- **Lesson 27.2: Fenwick Tree Applications**
 - Difficulty Level: Advanced
 - Learning Outcome: Use BIT for range sum, point update, and vice versa (range update, point query).
 - Topics Covered: Range update trick with BIT, getting range sum via two prefix queries.
 - Step-by-Step Algorithm Explanation: Compute $\text{sum}[l..r]$ as $\text{query}(r) - \text{query}(l-1)$.
 - Time Complexity: $O(\log n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Query sum 2..4 in $[1, 2, 3, 4]$ using BIT.
 - Visual Explanation: Overlapping ranges for prefix queries.
 - Practical Exercises: Solve “inversion count” using BIT.
 - Mini Assignment: Use BIT for frequency counting problems.
 - Common Mistakes: Forget to update both endpoints for range update trick.
 - Interview Questions: “How to get prefix sum using BIT?”
 - Quiz Questions: Calculation of prefix sum from BIT.

Chapter 28: Sparse Table

- **Learning Objective:** Learn sparse table for static range queries (idempotent functions like min, gcd) with $O(n \log n)$ build and $O(1)$ query.
- **Skills Covered:** Precompute powers of two ranges; answering range min/gcd queries.
- **Prerequisites:** DP (Chapter 25), log table knowledge.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 28.1: Building Sparse Table**

- Difficulty Level: Intermediate
- Learning Outcome: Precompute sparse table for range queries (min or gcd).
- Topics Covered: Table of size $[n][\log n]$, filling with interval lengths.
- Step-by-Step Algorithm Explanation: $dp[i][0] = arr[i]$; $dp[i][j] = f(dp[i][j-1], dp[i+2^{(j-1)}][j-1])$.
- Time Complexity: $O(n \log n)$.
- Space Complexity: $O(n \log n)$.
- Dry Run: Build for $arr=[1,3,2,5]$.
- Visual Explanation: Overlapping intervals depiction.
- Practical Exercises: Compute log table and sparse table for small array.
- Mini Assignment: Preprocess for range gcd queries.
- Common Mistakes: Indexing out of bounds for intervals.
- Interview Questions: “When to use sparse table vs segment tree?”
- Quiz Questions: Value of $sparse_table[2][1]$ given table values.
- **Lesson 28.2: Sparse Table Queries**
 - Difficulty Level: Intermediate
 - Learning Outcome: Answer any idempotent range query in $O(1)$ using sparse table.
 - Topics Covered: Using precomputed table to answer range min/gcd.
 - Step-by-Step Algorithm Explanation: Query $[L,R]$: length = k , return $\min(dp[L][k], dp[R-2^k+1][k])$.
 - Time Complexity: $O(1)$.
 - Space Complexity: $O(n \log n)$.
 - Dry Run: Query min in $[1,3]$ on example table.
 - Visual Explanation: Two overlapping range examples.
 - Practical Exercises: Use sparse table for RMQ (range minimum query).
 - Mini Assignment: Answer several preprocessed range queries.
 - Common Mistakes: Wrong interval length calculation.
 - Interview Questions: “What queries can sparse table handle?”
 - Quiz Questions: Range min result using given sparse entries.

Chapter 29: Monotonic Stack

- **Learning Objective:** Learn the monotonic stack pattern for next/previous greater/smaller element problems, solving many array and histogram problems in $O(n)$.
- **Skills Covered:** Maintaining a stack that keeps elements in sorted order, solving problems like largest rectangle in histogram, sliding window max.
- **Prerequisites:** Stack (Chapter 15).
- **Estimated Learning Time:** 2–3 hours.

- **Number of Lessons: 2**
- **Lesson 29.1: Next/Previous Greater Element**
 - Difficulty Level: Intermediate
 - Learning Outcome: Find next or previous greater/smaller element for each position in array.
 - Topics Covered: Stack-based one-pass algorithm.
 - Step-by-Step Algorithm Explanation: Process elements, pop smaller until stack top > current.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Next greater for [2,1,2,4,3].
 - Visual Explanation: Stack content evolution.
 - Practical Exercises: Code next greater/smaller element.
 - Mini Assignment: Largest rectangle in histogram using previous/next smaller.
 - Common Mistakes: Not clearing stack after loop; forgetting to push current.
 - Interview Questions: “Explain monotonic stack idea.”
 - Quiz Questions: Next greater indices in sample array.
- **Lesson 29.2: Monotonic Stack Applications**
 - Difficulty Level: Advanced
 - Learning Outcome: Solve problems like “largest rectangle in histogram” and “max sliding window” using monotonic stack/queue.
 - Topics Covered: Use stack to compute max area under histogram in $O(n)$.
 - Step-by-Step Algorithm Explanation: Histogram largest rectangle example on [2,1,5,6,2,3].
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Expand rectangle from each bar to nearest smaller.
 - Visual Explanation: Bar spans illustrated on histogram.
 - Practical Exercises: Max sliding window (can also use deque) via stack logic.
 - Mini Assignment: Largest square in histogram variant.
 - Common Mistakes: Off-by-one in width calculation.
 - Interview Questions: “How to find largest rectangle under skyline?”
 - Quiz Questions: Rectangle area for given bar and neighbors.

Chapter 30: Monotonic Queue

- **Learning Objective:** Learn the monotonic queue pattern for sliding-window problems where window values need ordering (like max/min).

- **Skills Covered:** Maintain double-ended queue in monotonic order; solve sliding window max/min in $O(n)$.
- **Prerequisites:** Queue (Chapter 16).
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 30.1: Sliding Window Maximum**
 - Difficulty Level: Intermediate
 - Learning Outcome: Solve maximum in each sliding window in $O(n)$ using a deque.
 - Topics Covered: Maintain deque of candidates in decreasing order.
 - Step-by-Step Algorithm Explanation: For each index, pop smaller elements from back, remove old from front.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(k)$.
 - Dry Run: Window max with $[1, 3, -1, -3, 5, 3, 6, 7]$, $k=3$.
 - Visual Explanation: Deque content over time.
 - Practical Exercises: Code sliding window maximum.
 - Mini Assignment: Sliding window minimum variant.
 - Common Mistakes: Forget to remove elements out of window at front.
 - Interview Questions: “Explain deque use for sliding window.”
 - Quiz Questions: Deque state after processing first 5 elements.
- **Lesson 30.2: Deque Applications**
 - Difficulty Level: Advanced
 - Learning Outcome: Apply monotonic queue to more problems: sum of minimums, traffic data smoothing.
 - Topics Covered: Use deque for streaming order-statistics (min, max).
 - Step-by-Step Algorithm Explanation: Compute sliding window min for $[2, 1, 5, 3]$.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(k)$.
 - Dry Run: Window min for $[4, 2, 12, 3]$ $k=2$.
 - Visual Explanation: Monotonic increase queue content.
 - Practical Exercises: “Minimum Sliding Window”, “Max of subarrays”.
 - Mini Assignment: Compute sum of sliding window minimums.
 - Common Mistakes: Re-using deque without clearing between runs.
 - Interview Questions: “What data structure for sliding max?”
 - Quiz Questions: Front of deque after processing sample.

Chapter 31: Advanced Graph Algorithms

- **Learning Objective:** Explore complex graph algorithms: A* search (if time), Advanced MST (Borůvka's), network flow basics (Ford-Fulkerson, Edmonds-Karp).
- **Skills Covered:** Algorithmic variety: A* heuristic (concept), Borůvka's algorithm, Max flow algorithm understanding.
- **Prerequisites:** Graphs, greedy (Chapter 23,24).
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 4
- **Lesson 31.1: A* Search (Optional)**
 - Difficulty Level: Advanced
 - Learning Outcome: Introduce heuristic search (A*).
 - Topics Covered: $f = g + h$ cost, admissible heuristics.
 - Step-by-Step Algorithm Explanation: Example grid pathfinding with Manhattan distance heuristic.
 - Time Complexity: Dependent on heuristic; up to $O(b^d)$.
 - Space Complexity: $O(\text{nodes visited})$.
 - Dry Run: Path search on small grid.
 - Visual Explanation: Nodes expanded vs heuristic estimate.
 - Practical Exercises: Compare BFS vs A* performance.
 - Mini Assignment: Implement A* on simple weighted maze.
 - Common Mistakes: Non-admissible heuristic leads to wrong result.
 - Interview Questions: "What is A* and when use it?"
 - Quiz Questions: Heuristic values for sample node.
- **Lesson 31.2: Borůvka's MST Algorithm**
 - Difficulty Level: Advanced
 - Learning Outcome: Learn Borůvka's method for MST: repeatedly add cheapest edge from each component.
 - Topics Covered: Component detection with DSU, merging process.
 - Step-by-Step Algorithm Explanation: One iteration selecting cheapest edges for each tree.
 - Time Complexity: $O(E \log V)$.
 - Space Complexity: $O(V)$.
 - Dry Run: Step through MST edge additions.
 - Visual Explanation: Components merging.
 - Practical Exercises: Compare with Kruskal/Prim on same graph.
 - Mini Assignment: Implement one Borůvka pass with DSU.
 - Common Mistakes: Not updating component labels after merge.

- Interview Questions: “Explain Borůvka’s MST in simple terms.”
- Quiz Questions: Next edge chosen by Borůvka.
- **Lesson 31.3: Ford-Fulkerson & Edmonds-Karp (Max Flow)**
 - Difficulty Level: Advanced
 - Learning Outcome: Find maximum flow in network using augmenting paths and BFS (Edmonds-Karp).
 - Topics Covered: Residual graph, augmenting path search, updating flows.
 - Step-by-Step Algorithm Explanation: Ford-Fulkerson on small graph.
 - Time Complexity: $O(E * \text{max_flow})$ naive; Edmonds-Karp $O(VE^2)$.
 - Space Complexity: $O(E+V)$.
 - Dry Run: Augment path in sample network flow.
 - Visual Explanation: Residual capacity updates.
 - Practical Exercises: Compute max flow of a given network by hand.
 - Mini Assignment: Implement Edmonds-Karp with BFS for augmenting path.
 - Common Mistakes: Not updating reverse edges.
 - Interview Questions: “What is max flow-min cut theorem?”
 - Quiz Questions: Next augmentation path in example.
- **Lesson 31.4: Network Flow Applications**
 - Difficulty Level: Advanced
 - Learning Outcome: Apply max flow to bipartite matching, min cut problems.
 - Topics Covered: Transform bipartite matching to flow, minimum cut extraction.
 - Step-by-Step Algorithm Explanation: Reduce matching to flow network.
 - Time Complexity: Same as max flow.
 - Space Complexity: $O(E)$.
 - Dry Run: Match students to jobs via flow.
 - Visual Explanation: Flow network construction.
 - Practical Exercises: Solve bipartite matching with flow.
 - Mini Assignment: Use min-cut for segmentation example.
 - Common Mistakes: Not adding reverse edges in conversion.
 - Interview Questions: “How to model bipartite matching as flow?”
 - Quiz Questions: Matching output from a small bipartite graph.

Chapter 32: String Algorithms

- **Learning Objective:** Advanced string algorithms beyond basic matching: KMP (linear pattern search), Rabin-Karp (rolling hash), Z-algorithm.
- **Skills Covered:** Failure function (KMP), rolling hash for substring search, Z-array computation.

- **Prerequisites:** Strings (Chapter 6), hashing.
- **Estimated Learning Time:** 4–5 hours.
- **Number of Lessons:** 3
- **Lesson 32.1: Knuth–Morris–Pratt (KMP)**
 - Difficulty Level: Advanced
 - Learning Outcome: Perform fast string search by precomputing longest proper prefix-suffix (lps) array.
 - Topics Covered: LPS computation, pattern search using lps to skip re-checks.
 - Step-by-Step Algorithm Explanation: Build lps for “ABABCABAB”, then search in text.
 - Time Complexity: $O(n+m)$.
 - Space Complexity: $O(m)$ for lps.
 - Dry Run: KMP search example.
 - Visual Explanation: LPS table vs pattern shifts.
 - Practical Exercises: Implement KMP lps array and search.
 - Mini Assignment: Find all occurrences of pattern in text.
 - Common Mistakes: Off-by-one in lps loop.
 - Interview Questions: “Explain KMP in one sentence.”
 - Quiz Questions: LPS value at each index for sample pattern.
- **Lesson 32.2: Rabin-Karp**
 - Difficulty Level: Intermediate
 - Learning Outcome: Use rolling hash to find substring matches with average linear time.
 - Topics Covered: Polynomial rolling hash, handling collisions via double-check.
 - Step-by-Step Algorithm Explanation: Compute hash for pattern and slide over text, compare on hash match.
 - Time Complexity: Average $O(n+m)$, worst $O(n*m)$ due to collisions.
 - Space Complexity: $O(1)$.
 - Dry Run: Example rolling hash on “abc”.
 - Visual Explanation: Hash window sliding over characters.
 - Practical Exercises: Implement Rabin-Karp for pattern search.
 - Mini Assignment: Compute hash of “hello” with base 26.
 - Common Mistakes: Overflow in hash, not using mod.
 - Interview Questions: “Why use mod in rolling hash?”
 - Quiz Questions: Next hash value after sliding window.
- **Lesson 32.3: Z-Algorithm**

- Difficulty Level: Advanced
- Learning Outcome: Compute Z-array for a string to allow substring search and pattern matching.
- Topics Covered: Z-box and Z-array, using Z for pattern in text (concat with separator).
- Step-by-Step Algorithm Explanation: Calculate Z for “aabxaabxcaabxaabxay”.
- Time Complexity: $O(n)$.
- Space Complexity: $O(n)$.
- Dry Run: Compute Z-array for short example.
- Visual Explanation: Z-box window moving.
- Practical Exercises: Use Z to find pattern in text (prepend pattern + special + text).
- Mini Assignment: Code Z-array construction.
- Common Mistakes: Overlapping Z-box management.
- Interview Questions: “Difference between Z and KMP algorithms?”
- Quiz Questions: Value of Z at a given position for a string.

Chapter 33: Advanced Trees

- **Learning Objective:** Understand self-balancing trees (concept): AVL, Red-Black, and B-Tree structures, to appreciate why BST can be kept balanced.
- **Skills Covered:** Tree rotations (AVL), properties of Red-Black trees, B-tree node occupancy, though implementation detail is conceptual.
- **Prerequisites:** Binary Search Tree (Chapter 19).
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 33.1: AVL Trees (Concept)**
 - Difficulty Level: Advanced
 - Learning Outcome: Learn how AVL trees maintain balance (height difference ≤ 1) via rotations.
 - Topics Covered: AVL tree insert, balance factor, LL/LR/RL/RR rotations.
 - Step-by-Step Algorithm Explanation: Perform LL rotation on unbalanced node.
 - Time Complexity: $O(\log n)$ for insert/delete.
 - Space Complexity: $O(n)$.
 - Dry Run: Insert 3,2,1 causing LL rotation.
 - Visual Explanation: Before/after rotation illustration.
 - Practical Exercises: Trace heights after rotation on sample tree.
 - Mini Assignment: Implement single rotation functions.

- Common Mistakes: Applying wrong rotation type.
- Interview Questions: “What is an AVL tree?”
- Quiz Questions: Rotation needed when left subtree height is 2 greater.
- **Lesson 33.2: Red-Black Trees (Concept)**
 - Difficulty Level: Advanced
 - Learning Outcome: Understand Red-Black tree properties (roughly balanced via color rules) without coding detail.
 - Topics Covered: Node colors, properties (root black, red node children black, same # blacks on paths).
 - Step-by-Step Algorithm Explanation: Insert fix-up steps by color flips/rotations (conceptual).
 - Time Complexity: $O(\log n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Insert causing red-red conflict resolution.
 - Visual Explanation: Color change example on small tree.
 - Practical Exercises: Check if a given colored tree is a valid RBT.
 - Mini Assignment: Compare RBT vs AVL pros/cons.
 - Common Mistakes: Missing one of RB rules.
 - Interview Questions: “Why use Red-Black tree?”
 - Quiz Questions: Given a tree coloring, identify violations.
- **Lesson 33.3: B-Trees (Concept)**
 - Difficulty Level: Advanced
 - Learning Outcome: Learn B-tree insertion/deletion basics (used in databases/filesystems).
 - Topics Covered: Node with multiple keys/children, splitting/merging nodes, minimum degree.
 - Step-by-Step Algorithm Explanation: Insert into leaf and split if overflow.
 - Time Complexity: $O(\log n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: Insert keys causing a node split.
 - Visual Explanation: B-tree node structure before/after split.
 - Practical Exercises: Simulate insertion in B-tree of order 3.
 - Mini Assignment: Calculate height of B-tree with given branching factor.
 - Common Mistakes: Confusing B-tree with BST.
 - Interview Questions: “What is a B-tree and where is it used?”
 - Quiz Questions: Structure of B-tree node after insert sequence.

Chapter 34: Advanced Problem Solving

- **Learning Objective:** Combine multiple concepts to solve very challenging problems (e.g., use DSA + math). Practice higher-order thinking and creativity in problem solving.
- **Skills Covered:** Strategy for tackling unknown problem types, reading research papers or editorials, strategic reduction to known problems.
- **Prerequisites:** All previous chapters.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 34.1: Complex Coding Strategies**
 - Difficulty Level: Advanced
 - Learning Outcome: Learn strategies like meet-in-the-middle, pruning search, heuristic solutions.
 - Topics Covered: Examples of meet-in-the-middle in subset sum, use of heuristic approximations.
 - Step-by-Step Algorithm Explanation: Split problem into halves and combine results.
 - Time Complexity: $O(2^{(n/2)})$.
 - Space Complexity: $O(2^{(n/2)})$.
 - Dry Run: Meet-in-middle for sum=0 in array of size 6 by splitting into two halves.
 - Visual Explanation: Partitioned subsets chart.
 - Practical Exercises: Implement small meet-in-middle search.
 - Mini Assignment: Solve N up to 30 by splitting.
 - Common Mistakes: Memory explosion, duplicate handling.
 - Interview Questions: “When to use meet-in-the-middle?”
 - Quiz Questions: Steps to meet-in-middle technique on sample.
- **Lesson 34.2: Code Optimization and Patterns**
 - Difficulty Level: Advanced
 - Learning Outcome: Optimize code for time/space (bitsets, greedy approximations, caching).
 - Topics Covered: Low-level optimizations, bitset DP, caching heuristics.
 - Step-by-Step Algorithm Explanation: Using bit operations to speed up subset checks.
 - Time Complexity: Varies.
 - Space Complexity: Varies.
 - Dry Run: Optimize a nested loop by bitset.
 - Visual Explanation: Bitset bit operations.

- Practical Exercises: Convert $O(n^3)$ into bitset-friendly.
- Mini Assignment: Use caching (LRU) to avoid recomputation.
- Common Mistakes: Over-optimizing prematurely.
- Interview Questions: “How to speed up DP with bitsets?”
- Quiz Questions: Interpret bitset operations on example.
- **Lesson 34.3: Problem Decomposition**
 - Difficulty Level: Advanced
 - Learning Outcome: Break down large problems into subproblems; use known algorithms creatively.
 - Topics Covered: Reduce problem constraints, transform problem domain (coordinate compression, graph mapping).
 - Step-by-Step Algorithm Explanation: Example: reduce 2D problem via coordinate compression.
 - Time Complexity: Depends on transformation.
 - Space Complexity: Increased by mapping.
 - Dry Run: Compress coordinates [1000, 5000] to [0, 1].
 - Visual Explanation: Mapping points to grid.
 - Practical Exercises: Solve a scheduling problem by transforming times.
 - Mini Assignment: Apply graph modeling to solve a grid problem.
 - Common Mistakes: Incorrect mapping of indices.
 - Interview Questions: “What is coordinate compression?”
 - Quiz Questions: Outcome after compression on sample values.

Chapter 35: Coding Patterns Overview

- **Learning Objective:** Learn common solution patterns (not tied to data structure) for interview coding: e.g., sliding window, two-pointers, fast/slow pointers, interval merging.
- **Skills Covered:** Recognize problem patterns (in addition to data structure patterns above).
- **Prerequisites:** Chapters covering each pattern concept.
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 35.1: Overview of Coding Patterns**
 - Difficulty Level: Intermediate
 - Learning Outcome: Identify when to apply each pattern (two-pointer, sliding window, fast-slow, interval).
 - Topics Covered: Summary of patterns, classification of problems.
 - Step-by-Step Algorithm Explanation: N/A (overview).

- Time Complexity: Varies by pattern (often $O(n)$).
- Space Complexity: Typically $O(1)$ or $O(n)$.
- Dry Run: N/A.
- Visual Explanation: Table mapping patterns to example problems.
- Practical Exercises: Categorize sample problems by pattern.
- Mini Assignment: Given a new problem, decide on likely pattern to try.
- Common Mistakes: Jumping to code without recognizing pattern.
- Interview Questions: “What patterns do you know for sliding window problems?”
- Quiz Questions: Classify problems by pattern category.

Coding Pattern Chapters

Chapter 36: Two Pointer Pattern

- **Learning Objective:** (See Chapter 10 for techniques) This chapter consolidates pattern-based examples specifically labeled “Two Pointer Pattern.”
- **Skills Covered:** Applying two-pointer logic to variant problems (pair sum, palindrome, cycle detection).
- **Prerequisites:** Chapter 10.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 36.1: Two-Pointer Problems**
 - Difficulty Level: Intermediate
 - Learning Outcome: Recognize and implement the two-pointer pattern in various contexts.
 - Topics Covered: Sorted array pair sums, array palindrome check, find duplicates in sorted array.
 - Step-by-Step Algorithm Explanation: Use pointers to check sum or palindrome.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: (see earlier examples)
 - Visual Explanation: (two pointers diagram)
 - Practical Exercises: Classic two-pointer LeetCode problems.
 - Mini Assignment: Merge two sorted lists with two pointers.
 - Common Mistakes: Overlapping pointers.
 - Interview Questions: Reference two-pointer typical question.
 - Quiz Questions: Variation of two-sum steps.
- **Lesson 36.2: Fast & Slow Pointer Pattern**

- Difficulty Level: Intermediate
- Learning Outcome: Use two pointers at different speeds to detect cycles or find midpoints.
- Topics Covered: Floyd's cycle detection, finding middle of list, array loop detection.
- Step-by-Step Algorithm Explanation: Demonstrate tortoise-hare in list [1→2→3→2].
- Time Complexity: $O(n)$.
- Space Complexity: $O(1)$.
- Dry Run: (cycle detection).
- Visual Explanation: (diagram of pointers).
- Practical Exercises: Cycle detection in linked list.
- Mini Assignment: Find start of cycle.
- Common Mistakes: Off-by-one pointer increments.
- Interview Questions: "What is Floyd's algorithm?"
- Quiz Questions: Behavior of fast/slow pointers on an acyclic list.

Chapter 37: Sliding Window Pattern

- **Learning Objective:** (See Chapters 10-11) Reinforce sliding window problems in one dedicated chapter.
- **Skills Covered:** Identifying window problems, fixed/variable windows.
- **Prerequisites:** Chapters 10-11.
- **Estimated Learning Time:** 2 hours.
- **Number of Lessons:** 1
- **Lesson 37.1: Sliding Window Pattern Examples**
 - Difficulty Level: Intermediate
 - Learning Outcome: Apply sliding window systematically in different problems (sum, longest substring, etc.).
 - Topics Covered: Fixed vs variable, window sum, longest/shortest window under constraints.
 - Step-by-Step Algorithm Explanation: Example sliding window on string to find unique chars.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: (unique substring example)
 - Visual Explanation: (window on string).
 - Practical Exercises: Practice different sliding window scenarios.
 - Mini Assignment: Solve "minimum window substring".
 - Common Mistakes: Misplacing window resets.

- Interview Questions: “Give examples of sliding-window problems.”
- Quiz Questions: Identify window steps on sample.

Chapter 38: Fast & Slow Pointer Pattern

- **Learning Objective:** (Covered in 36) Short chapter with examples on fast/slow usage beyond cycle detection (e.g., linked list midpoint).
- **Skills Covered:** (see above)
- **Prerequisites:** Chapters 10, 17.
- **Estimated Learning Time:** 1 hour.
- **Number of Lessons:** 1
- **Lesson 38.1: Fast/Slow Patterns**
 - Difficulty Level: Beginner
 - Learning Outcome: Identify tasks solvable by two-speed pointers.
 - Topics Covered: Find middle of list/array, detect cycles.
 - Step-by-Step Algorithm Explanation: (midpoint of [1,2,3,4,5]).
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: (midpoint of odd-length list)
 - Visual Explanation: (pointer movement).
 - Practical Exercises: Find middle in one pass.
 - Mini Assignment: Check if linked list is palindrome by comparing two halves.
 - Common Mistakes: Not stopping correctly at middle.
 - Interview Questions: “How to find middle element of linked list efficiently?”
 - Quiz Questions: Position after each move for slow/fast pointers.

Chapter 39: Merge Intervals Pattern

- **Learning Objective:** Learn to merge overlapping intervals by sorting and scanning.
- **Skills Covered:** Sort by start, merge adjacent overlapping intervals.
- **Prerequisites:** Sorting (Chapter 8).
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 39.1: Merge Intervals**
 - Difficulty Level: Intermediate
 - Learning Outcome: Given a list of intervals, merge all overlapping ones.
 - Topics Covered: Sorting intervals by start, comparing current end with next start.

- Step-by-Step Algorithm Explanation: Merge [[1,3],[2,6],[8,10]] to [[1,6],[8,10]].
- Time Complexity: $O(n \log n)$.
- Space Complexity: $O(n)$.
- Dry Run: (above example).
- Visual Explanation: Timeline merging overlaps.
- Practical Exercises: Insert interval into existing merged list.
- Mini Assignment: Find gaps between intervals.
- Common Mistakes: Not updating end correctly, ignoring disjoint.
- Interview Questions: “Given sorted intervals, how to detect overlap?”
- Quiz Questions: Result of merge on sample set.

Chapter 40: Binary Search Pattern

- **Learning Objective:** Consolidate various binary search applications (midpoint search, search on answer, etc.).
- **Skills Covered:** (see Chapters 7, patterns)
- **Prerequisites:** Chapter 7.
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 40.1: Binary Search Applications**
 - Difficulty Level: Intermediate
 - Learning Outcome: Recognize when to apply binary search to find thresholds or answers.
 - Topics Covered: Finding minimum array element, first occurrence, last occurrence, search on answer.
 - Step-by-Step Algorithm Explanation: Find first index of $\geq$ target.
 - Time Complexity: $O(\log n)$.
 - Space Complexity: $O(1)$.
 - Dry Run: (first ≥ 5 in [1,3,5,7]).
 - Visual Explanation: (mid moving).
 - Practical Exercises: Find square root floor of a number via binary search.
 - Mini Assignment: “Split array largest sum” using binary on answer.
 - Common Mistakes: Infinite loops on equal pointers.
 - Interview Questions: “How to find boundary index with binary search?”
 - Quiz Questions: Next mid values for target search.

Chapter 41: Tree DFS Pattern

- **Learning Objective:** Use DFS patterns on trees to solve problems (e.g., backtracking on trees).

- **Skills Covered:** Recursive tree traversal patterns, path-building.
- **Prerequisites:** Tree traversals (Chapter 18).
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 41.1: Tree DFS Problems**
 - Difficulty Level: Intermediate
 - Learning Outcome: Apply DFS to trees for path-sum, tree paths enumeration.
 - Topics Covered: Accumulate path sums, collect paths.
 - Step-by-Step Algorithm Explanation: DFS with path list.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$ stack.
 - Dry Run: (path sum to 7 in small tree).
 - Visual Explanation: (paths highlighted in tree).
 - Practical Exercises: Sum of all root-to-leaf paths.
 - Mini Assignment: Serialize/deserialize tree with DFS.
 - Common Mistakes: Not backtracking path list.
 - Interview Questions: “How to list all root-to-leaf paths?”
 - Quiz Questions: DFS order outputs.

Chapter 42: Tree BFS Pattern

- **Learning Objective:** Use BFS (level order) on trees for problems (e.g., level order traversal, zigzag order).
- **Skills Covered:** Using queue for tree layers.
- **Prerequisites:** Tree traversal (Chapter 18).
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 42.1: Tree BFS Problems**
 - Difficulty Level: Intermediate
 - Learning Outcome: Apply BFS to group nodes by depth, solve tree puzzles.
 - Topics Covered: Level order traversal, connecting next pointers, snake order.
 - Step-by-Step Algorithm Explanation: BFS on example tree returning level sums.
 - Time Complexity: $O(n)$.
 - Space Complexity: $O(n)$.
 - Dry Run: (sum per level of [3,9,20,null,null,15,7]).
 - Visual Explanation: (tree layers colored).

- Practical Exercises: Populate next-right pointers using BFS.
- Mini Assignment: Zigzag level order printing.
- Common Mistakes: Losing track of level boundaries.
- Interview Questions: “How to traverse tree by levels?”
- Quiz Questions: Output levels from sample tree.

Chapter 43: Top K Elements Pattern

- **Learning Objective:** Solve top-k problems (largest/smallest) using heaps or quickselect.
- **Skills Covered:** Using priority queue to get top K, partial sorting (quickselect).
- **Prerequisites:** Heap, sorting.
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 43.1: Top K Elements**
 - Difficulty Level: Intermediate
 - Learning Outcome: Find k largest/smallest elements efficiently.
 - Topics Covered: Min-heap of size k, quickselect average-case $O(n)$.
 - Step-by-Step Algorithm Explanation: Maintain size-k min-heap for largest k.
 - Time Complexity: $O(n \log k)$.
 - Space Complexity: $O(k)$.
 - Dry Run: Top 3 of [4, 1, 7, 3, 9, 2].
 - Visual Explanation: Heap of size 3 with current top elements.
 - Practical Exercises: Kth largest element in stream.
 - Mini Assignment: Partition approach quickselect demonstration.
 - Common Mistakes: Using max-heap incorrectly for largest.
 - Interview Questions: “How to find kth largest element?”
 - Quiz Questions: Heap content after processing first 4 elements.

Chapter 44: K-Way Merge Pattern

- **Learning Objective:** Merge multiple sorted lists/arrays (k-way merge) using heap or divide-and-conquer.
- **Skills Covered:** Priority queue approach (merging k lists), recursive merging (pairwise).
- **Prerequisites:** Priority Queue, divide & conquer (Chapter 23).
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 44.1: K-Way Merge**

- Difficulty Level: Intermediate
- Learning Outcome: Merge k sorted lists into one sorted list.
- Topics Covered: Using min-heap of size k to pick next smallest, or merge lists pairwise.
- Step-by-Step Algorithm Explanation: Put first element of each list into heap and extract min repeatedly.
- Time Complexity: $O(n \log k)$ where n is total elements.
- Space Complexity: $O(k)$ for heap.
- Dry Run: Merge [1,4], [2,5,6], [3] steps.
- Visual Explanation: Heap of current heads of lists.
- Practical Exercises: Merge k sorted linked-lists (LeetCode “Merge k Lists”).
- Mini Assignment: Implement heap-based k-way merge on arrays.
- Common Mistakes: Off-by-one in pointers per list.
- Interview Questions: “What is complexity of merging k sorted arrays?”
- Quiz Questions: Next output element from example.

Chapter 45: Backtracking Pattern

- **Learning Objective:** Consolidate backtracking pattern in one chapter (see Chapter 4).
- **Skills Covered:** (As 4)
- **Prerequisites:** Chapter 4.
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 45.1: Backtracking Pattern Examples**
 - Difficulty Level: Intermediate
 - Learning Outcome: Practice backtracking on new problem types (matrix word search, subset sums).
 - Topics Covered: Problem-solving with backtracking.
 - Step-by-Step Algorithm Explanation: Word search II example using backtracking and pruning.
 - Time Complexity: High (exponential).
 - Space Complexity: $O(n)$ stack + output.
 - Dry Run: Find words in grid “A,B” example.
 - Visual Explanation: Search paths through grid.
 - Practical Exercises: Sudoku solver with backtracking.
 - Mini Assignment: Letter combinations of a phone number.
 - Common Mistakes: Not backtracking indexes correctly.
 - Interview Questions: “How do you prune in backtracking?”
 - Quiz Questions: Next recursive call for sample state.

Chapter 46: Dynamic Programming Pattern

- **Learning Objective:** (See Chapter 25) Emphasize DP pattern classification and problem mapping.
- **Skills Covered:** (See 25)
- **Prerequisites:** Chapter 25.
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 46.1: DP Pattern Problems**
 - Difficulty Level: Intermediate
 - Learning Outcome: Identify DP problem patterns (subsequence, subarray, knapsack, tree DP).
 - Topics Covered: (Examples covered before).
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: Varies.
 - Space Complexity: Varies.
 - Dry Run: N/A.
 - Visual Explanation: DP table examples.
 - Practical Exercises: Map new problems to known DP templates.
 - Mini Assignment: Classify problem as knapsack, LIS, etc.
 - Common Mistakes: Using DP unnecessarily where greedy suffices.
 - Interview Questions: “What DP categories do you know?”
 - Quiz Questions: Identify DP formula type for example recurrence.

Chapter 47: Graph Patterns

- **Learning Objective:** (See Chapter 23) Recap graph problem patterns (shortest path, connectivity).
- **Skills Covered:** (See 23)
- **Prerequisites:** Chapter 23.
- **Estimated Learning Time:** 1–2 hours.
- **Number of Lessons:** 1
- **Lesson 47.1: Graph Pattern Problems**
 - Difficulty Level: Intermediate
 - Learning Outcome: Recognize which graph algorithms to apply (BFS vs DFS vs Dijkstra).
 - Topics Covered: Connectivity problems, pathfinding, MST, cycle detection patterns.
 - Step-by-Step Algorithm Explanation: N/A.

- Time Complexity: Varies by algorithm.
- Space Complexity: Varies.
- Dry Run: N/A.
- Visual Explanation: Graph with annotated solution path.
- Practical Exercises: Given problem statement, choose appropriate graph approach.
- Mini Assignment: Convert grid to graph and search.
- Common Mistakes: Using BFS on weighted edges.
- Interview Questions: “When to use BFS vs DFS in graphs?”
- Quiz Questions: Map problems (e.g. “rotting oranges”) to BFS pattern.

Chapter 48: LeetCode Preparation

- **Learning Objective:** Practice a curated set of problems categorized by difficulty to solidify learning.
- **Skills Covered:** Solving easy, medium, and hard problems on coding platforms; test-taking strategies.
- **Prerequisites:** All DSA chapters.
- **Estimated Learning Time:** 8–10 hours.
- **Number of Lessons:** 3
- **Lesson 48.1: Easy Level Problems**
 - Difficulty Level: Beginner
 - Learning Outcome: Gain confidence through solving common easy problems (arrays, strings, math).
 - Topics Covered: Easy topics review (simple two sum, reverse string, BFS on small graph).
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: Varies (mostly $O(n)$).
 - Space Complexity: Varies.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Attempt 10-20 easy LeetCode problems (Blind 75 easy).
 - Mini Assignment: Track solved problems and time.
 - Common Mistakes: Overthinking easy problems, syntax errors.
 - Interview Questions: “(Review an easy problem from practice)”.
 - Quiz Questions: N/A.
- **Lesson 48.2: Medium Level Problems**
 - Difficulty Level: Intermediate

- Learning Outcome: Tackle more challenging problems requiring pattern application (sliding window, DP, trees).
- Topics Covered: Medium topics (LRU cache, graph traversal, DP on arrays).
- Step-by-Step Algorithm Explanation: N/A.
- Time Complexity: $O(n \log n)$ or $O(n^2)$ typical.
- Space Complexity: Varies.
- Dry Run: N/A.
- Visual Explanation: N/A.
- Practical Exercises: Solve 20-30 medium problems, including Blind 75 medium.
- Mini Assignment: Revise solutions and identify optimization.
- Common Mistakes: Partial solutions, not handling edge cases.
- Interview Questions: “(Discuss a medium-level problem approach)”.
- Quiz Questions: N/A.
- **Lesson 48.3: Hard Level Problems**
 - Difficulty Level: Advanced
 - Learning Outcome: Develop persistence on toughest problems, integrate multiple concepts.
 - Topics Covered: Hard patterns (graph DP, advanced backtracking).
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: Typically high; optimize if possible.
 - Space Complexity: Varies.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Solve 10-15 hard problems (some Blind 75 hard).
 - Mini Assignment: Break down problem into sub-tasks and solve.
 - Common Mistakes: Getting stuck and giving up early.
 - Interview Questions: “Explain your approach to problem X.”
 - Quiz Questions: N/A.

Chapter 49: Company-wise Preparation

- **Learning Objective:** Tailor preparation to specific company patterns and expectations (Google, Amazon, Microsoft, Meta, Apple, Netflix, Adobe, Atlassian, Uber, LinkedIn).
- **Skills Covered:** Understanding each company’s interview style, typical question types, behavioral focus.
- **Prerequisites:** All technical chapters, soft skills.
- **Estimated Learning Time:** 5–6 hours.
- **Number of Lessons:** 5

- **Lesson 49.1: Google Interview Strategy**

- Difficulty Level: N/A (strategic)
- Learning Outcome: Learn Google's emphasis (data structures fundamentals, whiteboard clarity, communication).
- Topics Covered: Google's interview stages, sample problem types, tips.
- Step-by-Step Algorithm Explanation: N/A.
- Time Complexity: N/A.
- Space Complexity: N/A.
- Dry Run: N/A.
- Visual Explanation: N/A.
- Practical Exercises: Mock Google interview problems (array, graph, DP).
- Mini Assignment: Prepare self-introduction and project pitch.
- Common Mistakes: Not explaining thinking clearly.
- Interview Questions: Focus on how Google evaluates (e.g., "Googlyness").
- Quiz Questions: N/A.

- **Lesson 49.2: Amazon & Microsoft Interview Strategies**

- Difficulty Level: N/A
- Learning Outcome: Amazon's leadership principles integration; Microsoft's product-focus and technical depth.
- Topics Covered: Amazon's question trends (system design, arrays), Microsoft's emphasis (CLR, algorithms).
- Step-by-Step Algorithm Explanation: N/A.
- Time Complexity: N/A.
- Space Complexity: N/A.
- Dry Run: N/A.
- Visual Explanation: N/A.
- Practical Exercises: Practice problems often asked by these companies.
- Mini Assignment: Relate projects to Amazon leadership principles.
- Common Mistakes: Not addressing Amazon principles in answers.
- Interview Questions: Company-specific scenarios.
- Quiz Questions: N/A.

- **Lesson 49.3: Meta (Facebook) & Apple Preparation**

- Difficulty Level: N/A
- Learning Outcome: Meta: Focus on algorithms and system design; Apple: emphasis on coding style and optimization.
- Topics Covered: Common patterns and recent trends (e.g., Facebook's emphasis on problem solving + system design for SDE2+).
- Step-by-Step Algorithm Explanation: N/A.

- Time Complexity: N/A.
- Space Complexity: N/A.
- Dry Run: N/A.
- Visual Explanation: N/A.
- Practical Exercises: Meta style on LeetCode, Apple-specific puzzles.
- Mini Assignment: Prepare examples of past leadership experiences (behavioral).
- Common Mistakes: Overlooking edge cases in code.
- Interview Questions: Company-specific coding and behavior.
- Quiz Questions: N/A.
- **Lesson 49.4: Netflix, Adobe, Atlassian, Uber Interviews**
 - Difficulty Level: N/A
 - Learning Outcome: Understand interview culture: Netflix values problem-solving creativity; Adobe/Atlassian values teamwork; Uber focuses on hard scaling problems.
 - Topics Covered: Problem domain tendencies, company values, on-site vs phone differences.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Solve problems reported from these companies (e.g., Uber – multithreading, big data).
 - Mini Assignment: Review company blog interviews.
 - Common Mistakes: Not asking questions about company.
 - Interview Questions: Sample from each company's style.
 - Quiz Questions: N/A.
- **Lesson 49.5: LinkedIn, Others**
 - Difficulty Level: N/A
 - Learning Outcome: Last company strategies, emphasis on certain topics (LinkedIn often likes trees/graphs).
 - Topics Covered: Known question sets, focus areas (e.g., data models).
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Practice LinkedIn questions.

- Mini Assignment: Mock interview with focus on company's domain (professional networks for LinkedIn).
- Common Mistakes: Not aligning experiences with company mission.
- Interview Questions: Company-specific examples.
- Quiz Questions: N/A.

Chapter 50: Mock Interviews

- **Learning Objective:** Simulate real coding interviews to practice under pressure and receive feedback.
- **Skills Covered:** Live problem-solving, thinking aloud, written/whiteboard coding, timing practice.
- **Prerequisites:** All technical preparation.
- **Estimated Learning Time:** 3–4 hours per session (multi-session recommended).
- **Number of Lessons:** 3
- **Lesson 50.1: Conducting Mock Coding Interviews**
 - Difficulty Level: Intermediate
 - Learning Outcome: Practice full mock interviews, use feedback to improve.
 - Topics Covered: Problem selection, interviewer role-play, feedback mechanism.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Schedule and do mock interviews with peers or mentors.
 - Mini Assignment: Record solution to problem and review it critically.
 - Common Mistakes: Ignoring feedback, not simulating environment (no IDE).
 - Interview Questions: "Time yourself solving an example problem."
 - Quiz Questions: N/A.
- **Lesson 50.2: Whiteboard Coding Practice**
 - Difficulty Level: Intermediate
 - Learning Outcome: Gain confidence coding on whiteboard/paper (or simple text editor).
 - Topics Covered: Writing code by hand, simplifying syntax issues, clean thought process.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.

- Dry Run: Solve a simple problem on paper and walk through it.
- Visual Explanation: N/A.
- Practical Exercises: Work through common problems using pen and paper.
- Mini Assignment: Flip your device/IDE off and code a solution on paper.
- Common Mistakes: Not writing legibly; skipping edge-case checks due to environment.
- Interview Questions: “Practice writing code without syntax highlighting.”
- Quiz Questions: N/A.
- **Lesson 50.3: Feedback and Iteration**
 - Difficulty Level: Intermediate
 - Learning Outcome: Learn to use feedback to iterate on solutions and presentation.
 - Topics Covered: Identifying weak areas, tracking progress, setting goals after mock.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Review mock interview notes and implement improvements.
 - Mini Assignment: Retake a previously failed problem with new strategy.
 - Common Mistakes: Not reflecting on mistakes.
 - Interview Questions: “What did you learn from your last mock?”
 - Quiz Questions: N/A.

Chapter 51: Resume for Product-Based Companies

- **Learning Objective:** Craft a resume that highlights technical skills and projects for tech interviews.
- **Skills Covered:** Resume formatting, writing bullet points with impact, including relevant projects and experience.
- **Prerequisites:** None technical.
- **Estimated Learning Time:** 2–3 hours.
- **Number of Lessons:** 2
- **Lesson 51.1: Resume Writing Basics**
 - Difficulty Level: Beginner
 - Learning Outcome: Learn how to present experience and skills clearly.

- Topics Covered: Structure (contact, summary, experience, education, projects), keywords.
- Step-by-Step Algorithm Explanation: N/A.
- Time Complexity: N/A.
- Space Complexity: N/A.
- Dry Run: N/A.
- Visual Explanation: Resume template example.
- Practical Exercises: Rewrite an existing resume section to be more concise.
- Mini Assignment: Get resume reviewed by mentor/peer.
- Common Mistakes: Typos, generic descriptions, no metrics.
- Interview Questions: “Walk me through your resume.”
- Quiz Questions: N/A.
- **Lesson 51.2: Tailoring for Tech Roles**
 - Difficulty Level: Beginner
 - Learning Outcome: Emphasize DSA/technical projects; align resume with product company expectations.
 - Topics Covered: Include GitHub links, coding challenge scores, relevant coursework, side projects.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: Example bullet “Implemented LRU cache in C++ (time O(1) ops).”
 - Practical Exercises: Add a technical project with problem solved.
 - Mini Assignment: Highlight 3 quantifiable achievements.
 - Common Mistakes: Listing irrelevant hobbies over skills.
 - Interview Questions: “Why did you include project X?”
 - Quiz Questions: N/A.

Chapter 52: Final Interview Preparation

- **Learning Objective:** Consolidate final tips for technical and behavioral interviews (coding practice, HR fundamentals).
- **Skills Covered:** Whiteboard etiquette, interview puzzles, behavioral question frameworks (STAR), negotiation basics.
- **Prerequisites:** All above.
- **Estimated Learning Time:** 3–4 hours.
- **Number of Lessons:** 3
- **Lesson 52.1: Coding Interview Wrap-Up**

- Difficulty Level: Intermediate
- Learning Outcome: Review last-minute checks (sleep well, resume copies, clarifying questions).
- Topics Covered: Rest, nutritional tips, reviewing key formulas/algorithms.
- Step-by-Step Algorithm Explanation: N/A.
- Time Complexity: N/A.
- Space Complexity: N/A.
- Dry Run: N/A.
- Visual Explanation: N/A.
- Practical Exercises: Mock final interview under timed conditions.
- Mini Assignment: Review own code for common pitfalls (off-by-one, null checks).
- Common Mistakes: Cramming all night, forgetting company-specific research.
- Interview Questions: “What do you bring to team/company?” (behavior).
- Quiz Questions: N/A.
- **Lesson 52.2: Behavioral Interviews Basics**
 - Difficulty Level: Beginner
 - Learning Outcome: Prepare for HR and behavioral rounds (STAR method).
 - Topics Covered: Common behavioral questions, formulating concise answers.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.
 - Space Complexity: N/A.
 - Dry Run: N/A.
 - Visual Explanation: N/A.
 - Practical Exercises: Answer mock behavioral questions.
 - Mini Assignment: Prepare stories for “teamwork”, “conflict”, “failure”.
 - Common Mistakes: Rambling, negativity.
 - Interview Questions: “Tell me about a time you faced X.”
 - Quiz Questions: N/A.
- **Lesson 52.3: Final Tips and Closing**
 - Difficulty Level: Beginner
 - Learning Outcome: Last advice on salary negotiation, sending thank-you notes, follow-up etiquette.
 - Topics Covered: Offer evaluation, negotiating equity/salary, multiple offers handling.
 - Step-by-Step Algorithm Explanation: N/A.
 - Time Complexity: N/A.

- Space Complexity: N/A.
- Dry Run: N/A.
- Visual Explanation: N/A.
- Practical Exercises: Draft a post-interview thank-you email.
- Mini Assignment: Research typical salary range for target role.
- Common Mistakes: Accepting first offer, not asking.
- Interview Questions: “What are your salary expectations?”
- Quiz Questions: N/A.

Practice Section

- **500+ Practice Questions:** Curate and categorize a large problem set by topic and difficulty.
- **Topic-wise Questions:** Lists of questions sorted by topic (arrays, strings, DP, etc.).
- **Company-wise Questions:** Collections specific to companies (e.g., “Amazon OA questions”).
- **Pattern-wise Questions:** Group problems by patterns (like “all sliding window problems”).
- **Blind 75:** Solve the popular “Blind 75” list of must-know problems.
- **Grind 75:** Include another curated set for practice.
- **NeetCode Roadmap:** Follow NeetCode’s topic list for structured practice.
- **Top 150 Interview Questions:** A compilation of frequently asked interview problems.

Projects

- **Library Management System:** Use data structures to manage books, search, sorting, hash tables for indexing.
- **Contact Manager:** Implement search/trie or hash for contacts.
- **Search Engine Simulation:** Build inverted index, use hashing and text algorithms (prefix trees).
- **Route Finder:** Use graphs (BFS/DFS) to find shortest path, implement Dijkstra.
- **File Compression (Huffman Coding):** Implement Huffman tree for basic text compression.
- **Social Network Graph:** Model connections as graph, implement friend recommendation.
- **Cache System (LRU):** Use hash + doubly linked list for LRU cache.
- **Task Scheduler:** Simulate task scheduling with priority queue.
- **Parking Management:** Use queue (parking lanes) and hashing (license lookup).
- **Online Judge Simulation:** Develop a mini coding judge (parsing, queueing, using data structures for limits).

Additional Resources and Strategies

- **Complete DSA Roadmap:** Diagram or list from basics to advanced topics (use outline above as guide).
- **Coding Interview Roadmap:** Steps from learning to applying (plan, practice, mock, polish).
- **Placement Preparation Strategy:** Timeline and milestones (learning, projects, resume, apply).
- **Problem Solving Strategy:** Clarify, plan, code, test—framework to approach.
- **Revision Plan:** Schedule to revisit each topic (DSA, puzzle, system design, etc.).
- **Daily Practice Schedule:** Hour-by-hour plan balancing coding, interview prep, rest.
- **Common Interview Mistakes:** List pitfalls (e.g., not clarifying, ignoring constraints).
- **Whiteboard Interview Tips:** Write legibly, structure code, iterate on solution.
- **Online Assessment Tips:** Familiarity with coding environments, reading instructions carefully.
- **HR Interview Basics:** Common HR questions, how to answer (STAR method).
- **Behavioral Interview Basics:** Techniques to discuss experiences, show cultural fit.
- **Coding Best Practices:** Clean code, naming, commenting, edge-case handling.
- **Complexity Cheat Sheet:** Quick reference Big-O of common operations and algorithms.
- **DSA Cheat Sheet:** Key formulas and structures (like prefix/suffix ideas, recursion pattern).
- **300+ Interview Questions:** Extended compilation beyond practice sets.
- **Best Books:** “Algorithms” (Sedgewick), “Cracking the Coding Interview”, etc.
- **Official Documentation:** Links to language docs (C++, Java, Python) and algorithm references.
- **GitHub Repositories:** Lists of useful code libraries and interview practice repos.
- **Coding Platforms:** LeetCode, Codeforces, HackerRank, CodeChef etc.
- **Recommended Learning Resources:** Blogs (GeeksforGeeks, CLRS), courses (Coursera, Udemy), YouTube channels.

Sources: Authoritative interview preparation guides and curriculum outlines were consulted to design this roadmap **【12†L114-L122】 【8†L89-L94】 【18†L52-L60】** . Each chapter and lesson follows industry best practices for SDE interview training.
